Lab Protocol Security Exercise

Abstract

We are trying to harden a WordPress web server environment on Debian 12 (netinstall ISO). We use Wordpress with Nginx, PHP-FPM and MariaDB. Our idea is to harden the os (Firewall, SSH, Updates) and the applications themselves (webserver, database, WordPress). For this we look at three different stages: default right after the Debian installation, baseline after installing all the necessary applications and then after hardening and look at the differences from a security angle. As tools we want to use Lynis, WPScan, nmap and ZAP. Our goal is to reduce vulnerabilities and attack surfaces and to make our security measurable with trackable metrics.

Contents

Abstract 1
1 Procedure and Set-Up 2
Network table 3
Installed services / packages 3
Questions asked 3
2 Analysis Part 4
Default Baseline 4
OS Hardening & Setup 6
WordPress Baseline (before hardening) 9
WordPress hardening 14
Webserver Hardening 15
PHP-FPM, MariaDB, AppArmor, auditd, Backup 21
After Hardening Tests 27
3 Conclusions 37
4 References 38
Course Materials 38
WordPress 39
Nginx 39
MariaDB 39
Debian 39
Tools 39
5 Appendix 39

1 Procedure and Set-Up

Wordpress Web Application scenario
OS: Debian 12 Bookworm (suggestion in hints & scenarios), netinstall ISO, because we want only things we really need, only ssh server and standard system utilities + grub boot loader
Virtualization: VirtualBox 7.2.8 on Windows 11
RAM: 4 GB, 2 CPUs, 20 G Disk
Network: Adapter 1 NAT (Internet), Adapter 2 host-only (SSH access, so that we both can access)

The first thing we do is to set up a plan of what we want to do and in what order so that we can follow a bit of a structure. The idea is to begin with installing Debian and look at the default baseline right away. Then we harden the OS (SSH, firewall, updates) and install and configure WordPress before capturing the second baseline. We continue to harden each layer systematically (webserver, PHP-FPM, database, AppArmor, logging and backup). Finally we run tests to compare them with the two baselines before to verify the effectiveness of our measures.

We install Debian 12 Bookworm as minimal server installation by using the netinstall ISO. To minimize the installed packages, we only chose “SSH server” so that both of us can access the vm and “standard system utilities”. We take a snapshot to be able to revisit this point in time.

To get an overview of how good our measurements really improve the security we use the tool lynis. We can also use the suggestions so that we know what to look for.

We choose Nginx as a web server because it doesn’t need so much storage and there are more configuration options than Apache. Nginx alone is not able to run php, that’s why we also get PHP-FPM. We need php-mysql to be able to work with MariaDB, php-curl, php-gd and php-mgstring so that WordPress functions correctly. As a database we use MariaDB locally.

We add a WordPress user account that can not log in, doesn’t have a password and can not use a shell. It is only a system user that owns WordPress files. That’s why we can also use wp-user as username and it is not unsafe.
sudo useradd -r -s /usr/sbin/nologin -d /var/www/wordpress wp-user
Then we download WordPress into /tmp, extract the files, create a new folder for WordPress and copy all of the files into the new folder.
We set wp-user as owner and add www-data as a group. This way all the data and files belong to wp-user. The webserver can read them but can not change anything.
sudo chown -R wp-user:www-data /var/www/wordpress

We don’t want to keep the unsafe mariadb default settings. That’s why we use the secure installation script.

We can open WordPress via Browser now. Username should not be admin to make it more secure. Search engine visibility is not needed, so deactivated.

Now we create another snapshot in VirtualBox to save everything before we start hardening. We also use a second VM to simulate attacks.

Network table

Host	IP	Role
wp-server	192.168.56.10	WordPress Server (Debian 12)
kali	192.168.56.102	Testmaschine

Installed services / packages

Package	Purpose
nginx	web server
php8.2-fpm	php processing
maridadb	database
fail2ban	intrusion prevention
ufw	firewall
unattended-upgrades	automatic security updates
auditd	audit logging
wp-cli	WordPress cli management
audispd-plugins	auditd plugin support
apparmor-profiles	additional AppArmor profiles
apparmor-profiles-extra	extra AppArmor profiles
needrestart	checks which services need restart after updates

Questions asked

Which services are running by default?
What is the baseline configuration of the minimal Debian 12 installation?
How can we reduce the attack surface of a WordPress installation through hardening?
How can we verify that our hardening measures actually work?

How much does the lynis score actually reflect the real security improvement?
How would the setup differ in a production environment with a real domain?

2 Analysis Part

Default Baseline

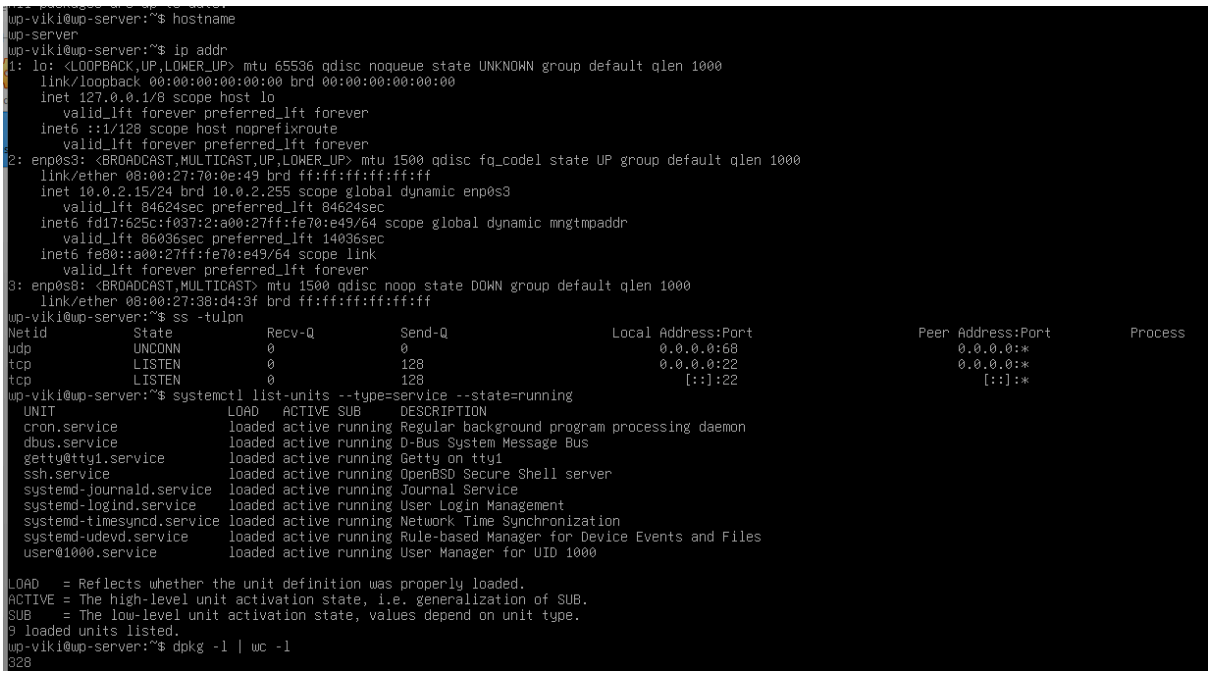
Sudo is not available after the installation, so we install it using “apt install sudo”. After that, we give our user sudo rights with “usermod -aG sudo” so that we don’t use the root for everything (least privilege) and that we better see in the logs who did what.

Before starting with anything else, we want to look at the default state so that we can see where potential problems might appear later and to see if the hardening works as expected. Additionally, we save a vm snapshot so that we can always come back to this state.

We use:
ip addr to see the network interfaces
ss -tulpn for the open ports
systemctl list-units for running services
dpkg -l | wc -l for packages
hostnamectl for all the important information of the system
timedatectl to check if the right timezone is active, important for logging
df -h for disk usage, important for storage planning
free -h for ram usage, important for ressource limits

Since the initial server installation is a netimage and we see that the running services are only the essential system services, we don’t identify anything we could delete without risking that something might not work as intended.

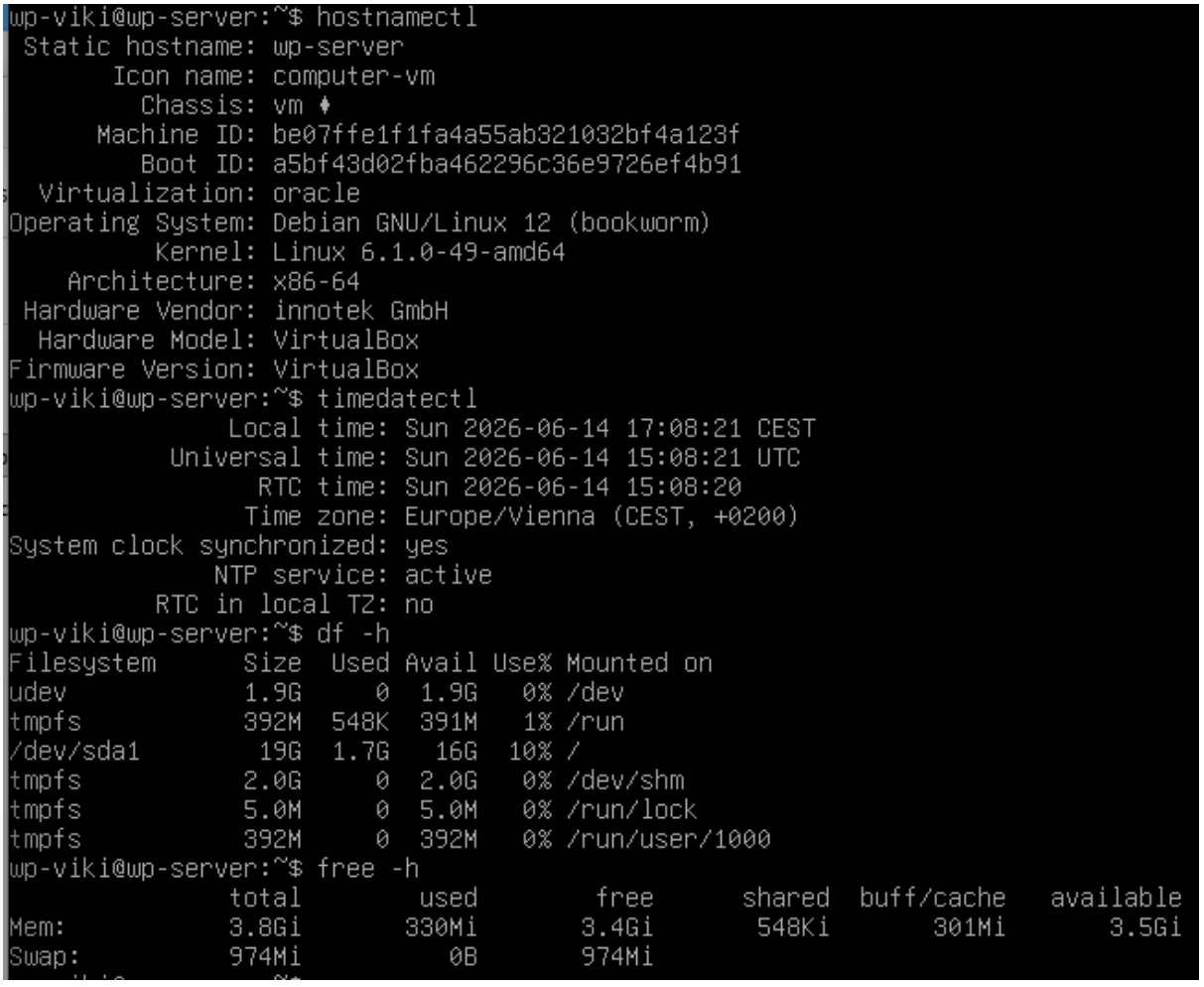
With getent passwd we see that there are only normal system users with nologin. Only root and our user have shell access. getent group shows us that our user is only in the standard groups that are automatically set through the user creation sudo, which we added through usermod before.



```
wp-viki@wp-server:~$ hostname
wp-server
wp-viki@wp-server:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:70:0e:49 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3
       valid_lft 84624sec preferred_lft 84624sec
    inet6 fd17:625c:f037:2:a00:27ff:fe70:e49/64 scope global dynamic mngtmpaddr
       valid_lft 86036sec preferred_lft 14036sec
    inet6 fe80::a00:27ff:fe70:e49/64 scope link
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether 08:00:27:38:d4:3f brd ff:ff:ff:ff:ff:ff
wp-viki@wp-server:~$ ss -tulpn
Netid          State           Recv-Q          Send-Q                    Local Address:Port                  Peer Address:Port         Process
udp            UNCONN          0               0                         0.0.0.0:68                         0.0.0.0:*
tcp            LISTEN          0               128                       0.0.0.0:22                         0.0.0.0:*
tcp            LISTEN          0               128                          [::]:22                            [::]:*
wp-viki@wp-server:~$ systemctl list-units --type=service --state=running
  UNIT                     LOAD   ACTIVE SUB     DESCRIPTION
  cron.service             loaded active running Regular background program processing daemon
  dbus.service             loaded active running D-Bus System Message Bus
  getty@tty1.service       loaded active running Getty on tty1
  ssh.service              loaded active running OpenBSD Secure Shell server
  systemd-journald.service loaded active running Journal Service
  systemd-logind.service   loaded active running User Login Management
  systemd-timesyncd.service loaded active running Network Time Synchronization
  systemd-udevd.service    loaded active running Rule-based Manager for Device Events and Files
  user@1000.service        loaded active running User Manager for UID 1000

LOAD   = Reflects whether the unit definition was properly loaded.
ACTIVE = The high-level unit activation state, i.e. generalization of SUB.
SUB    = The low-level unit activation state, values depend on unit type.
9 loaded units listed.
wp-viki@wp-server:~$ dpkg -l | wc -l
328
```

Figure 1: Output for default baseline ip addr, ss -tulpn, systemctl list, dpkg -l



```
wp-viki@wp-server:~$ hostnamectl
 Static hostname: wp-server
       Icon name: computer-vm
         Chassis: vm 🖴
      Machine ID: be07ffe1f1fa4a55ab321032bf4a123f
         Boot ID: a5bf43d02fba462296c36e9726ef4b91
  Virtualization: oracle
Operating System: Debian GNU/Linux 12 (bookworm)
          Kernel: Linux 6.1.0-49-amd64
    Architecture: x86-64
 Hardware Vendor: innotek GmbH
  Hardware Model: VirtualBox
Firmware Version: VirtualBox
wp-viki@wp-server:~$ timedatectl
               Local time: Sun 2026-06-14 17:08:21 CEST
           Universal time: Sun 2026-06-14 15:08:21 UTC
                 RTC time: Sun 2026-06-14 15:08:20
                Time zone: Europe/Vienna (CEST, +0200)
System clock synchronized: yes
              NTP service: active
          RTC in local TZ: no
wp-viki@wp-server:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            1.9G     0  1.9G   0% /dev
tmpfs           392M  548K  391M   1% /run
/dev/sda1        19G  1.7G   16G  10% /
tmpfs           2.0G     0  2.0G   0% /dev/shm
tmpfs           5.0M     0  5.0M   0% /run/lock
tmpfs           392M     0  392M   0% /run/user/1000
wp-viki@wp-server:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       330Mi       3.4Gi       548Ki       301Mi       3.5Gi
Swap:          974Mi          0B       974Mi
```

Figure 2: Output for default baseline hostnamectl, df -h, free -h

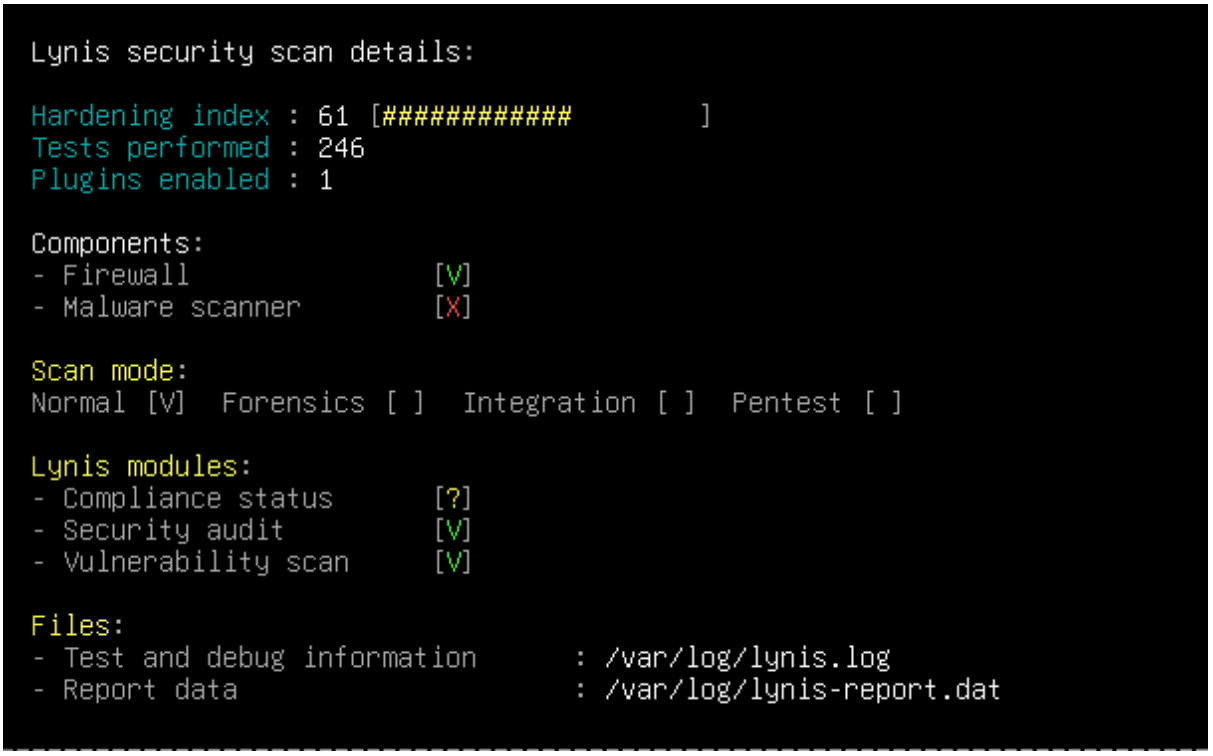


Figure 3: Output for default baseline Lynis scan 1

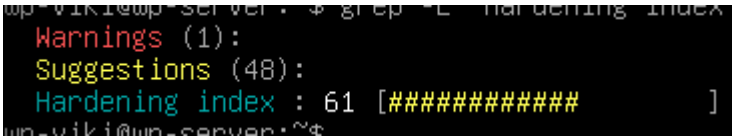


Figure 4: Output for default baseline Lynis scan 2

OS Hardening & Setup

We change the default ssh settings or set them active to:
PermitRootLogin no - it's harder to break in if you need to get both, username and password, with root the attacker would know the username already
PubkeyAuthentication yes - so that both of us can access
PasswordAuthentication yes - until we are done with the SSH settings
MaxAuthTries 3 - against brute force attacks
MaxSessions 2 - reduces misuse
AllowTcpForwarding no - SSH can not be used as a tunnel for other traffic (f.e. to skip firewall rules)
X11Forwarding no - we don't have a gui, unnecessary vulnerability

Before activating SSH, we set up the intrusion prevention tool fail2ban so that ip addresses will be blocked after using the wrong password several times. That helps against brute force attacks. Here we copy the standard configuration file jail.conf to jail.local as needed by fail2ban. Additionally, in the sshd-section of jail.local we add enabled = true and change backend to systemd so that fail2ban is able to find the logs.

To be able to access via SSH, we add to /etc/network/interfaces:
auto enp0s8

iface enp0s8 inet static
address 192.168.56.10
netmask 255.255.255.0
When connecting for the first time, we verify the SSH host key fingerprint to be sure we connected to the correct host. We stored the fingerprint securely for future reference.

```
wp-viki@wp-server:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     0
   `- Banned IP list:
```

Figure 5: Output fail2ban status

To set up the firewall, we install ufw (uncomplicated firewall) because the syntax of iptables is more complex and therefore more prone to errors. We change the rules to:
default deny incoming - we want to only allow specifically what we need
default allow outgoing - for example for updates
allow from 192.168.56.0/24 to any port 22 proto tcp - to access via SSH for both of us
allow 80/tcp - for http, necessary for the webserver
allow 443/tcp - for https, necessary for the webserver and TLS

```
wp-viki@wp-server:~$ sudo ufw show added
Added user rules (see 'ufw status' for running firewall):
ufw allow from 192.168.56.0/24 to any port 22 proto tcp
ufw allow 80/tcp
ufw allow 443/tcp
```

Figure 6: Firewall rules

```
wp-viki@wp-server:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    192.168.56.0/24
80/tcp                     ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
80/tcp (v6)                ALLOW IN    Anywhere (v6)
443/tcp (v6)               ALLOW IN    Anywhere (v6)
```

Figure 7: Firewall output after first rules

Known vulnerabilities are easy to access for attackers. So automatic updates are key to close them as fast and early as possible. Even if we are less in control of what exactly is installed and some updates might bring new bugs, the benefits are far bigger.
sudo apt install unattended-upgrades apt-listchanges
sudo dpkg-reconfigure --priority=low unattended-upgrades

```
wp-viki@wp-server:~$ apt-config dump APT::Periodic::Unattended-Upgrade
APT::Periodic::Unattended-Upgrade "1";
```

Figure 8: Automatic updates active

```
pkgs that look like they should be upgraded:
Fetched 0 B in 0s (0 B/s)
fetch.run() result: 0
Packages blacklist due to conffile prompts: []
No packages found that can be upgraded unattended and no pending auto-removals
The list of kept packages can't be calculated in dry-run mode.
```

Figure 9: Automatic updates work

sudo mariadb-secure-installation
Switch to unix_socket authentication: N - so that root also needs a password and scripts or other tools don't have problems accessing
Change the root password: Y - there is no default root password, so we definitely need one
Remove anonymous users: Y - no testuser that can access without password or username
Disallow root login remotely: Y - root access only locally, attackers using the network can not pretend to be root
Remove test database: Y - everyone could read or write in test database, we don't want that

We create a database with the settings recommended by WordPress inside mariadb through SQL commands.
CREATE DATABASE wordpress CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
We also create a db user that can only access locally. (password was changed to a stronger password of course)
CREATE USER 'wp-db-user'@'localhost' IDENTIFIED BY 'Passwort';
We give the user only the necessary rights for normal database usage, for updates that might change the database structure but only for the wordpress database.
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, INDEX, ALTER ON wordpress.* TO 'wp-db-user'@'localhost';

To be able to configure WordPress, first we copy the sample and then change the settings:
sudo cp /var/www/wordpress/wp-config-sample.php /var/www/wordpress/wp-config.php
define('DB_NAME', 'wordpress');
define('DB_USER', 'wp-db-user');
define('DB_PASSWORD', 'deinPasswort'); (we used a stronger password)
define('DB_HOST', 'localhost');
To get random and secure keys, we used https://api.wordpress.org/secret-key/1.1/salt/ and used those generated keys.

To configure the available sites in Nginx, we added to the wordpress file:

```
server {
    listen 80;
    server_name _;
    root /var/www/wordpress;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.2-fpm.sock;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

sudo ln -s /etc/nginx/sites-available/wordpress /etc/nginx/sites-enabled/ - to create a link between sites-available and sites-enabled
sudo rm /etc/nginx/sites-enabled/default - delete Nginx default page to be able to see WordPress instead

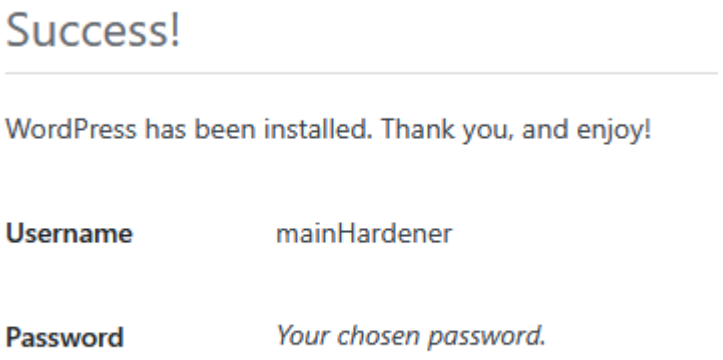


Figure 10: WordPress installed successfully

WordPress Baseline (before hardening)

```
wp-viki@wp-server:~$ ss -tulpn
Netid      State       Recv-Q      Send-Q           Local Address:Port            Peer Address:Port      Process
udp        UNCONN      0           0                      0.0.0.0:68                   0.0.0.0:*
tcp        LISTEN      0           80                   127.0.0.1:3306                 0.0.0.0:*
tcp        LISTEN      0           128                    0.0.0.0:22                   0.0.0.0:*
tcp        LISTEN      0           511                    0.0.0.0:80                   0.0.0.0:*
tcp        LISTEN      0           128                       [::]:22                      [::]:*
wp-viki@wp-server:~$ sudo systemctl --type=service --state=running
  UNIT                          LOAD   ACTIVE SUB     DESCRIPTION
  cron.service                  loaded active running Regular background program processing daemon
  dbus.service                  loaded active running D-Bus System Message Bus
  fail2ban.service              loaded active running Fail2Ban Service
  getty@tty1.service            loaded active running Getty on tty1
  mariadb.service               loaded active running MariaDB 10.11.14 database server
  nginx.service                 loaded active running A high performance web server and a reverse proxy server
  php8.2-fpm.service            loaded active running The PHP 8.2 FastCGI Process Manager
  ssh.service                   loaded active running OpenBSD Secure Shell server
  systemd-journald.service      loaded active running Journal Service
  systemd-logind.service        loaded active running User Login Management
  systemd-timesyncd.service     loaded active running Network Time Synchronization
  systemd-udevd.service         loaded active running Rule-based Manager for Device Events and Files
  unattended-upgrades.service loaded active running Unattended Upgrades Shutdown
  user@1000.service             loaded active running User Manager for UID 1000

LOAD   = Reflects whether the unit definition was properly loaded.
ACTIVE = The high-level unit activation state, i.e. generalization of SUB.
SUB    = The low-level unit activation state, values depend on unit type.
14 loaded units listed.
```

Figure 11: Baseline before hardening ss -tulpn, systemctl

lynis audit

```
Lynis security scan details:

Hardening index : 69 [##############      ]
Tests performed : 259
Plugins enabled : 1

Components:
- Firewall               [V]
- Malware scanner        [X]

Scan mode:
Normal [V]  Forensics [ ]  Integration [ ]  Pentest [ ]

Lynis modules:
- Compliance status      [?]
- Security audit         [V]
- Vulnerability scan     [V]

Files:
- Test and debug information      : /var/log/lynis.log
- Report data                     : /var/log/lynis-report.dat
```

Figure 12: Output Lynis scan before hardening 1

```
p-viki@wp-server:~$ grep -E 'Hardenin
 Warnings (1):
 Suggestions (45):
 Hardening index : 69 [##############
```

Figure 13: Output Lynis scan before hardening 2

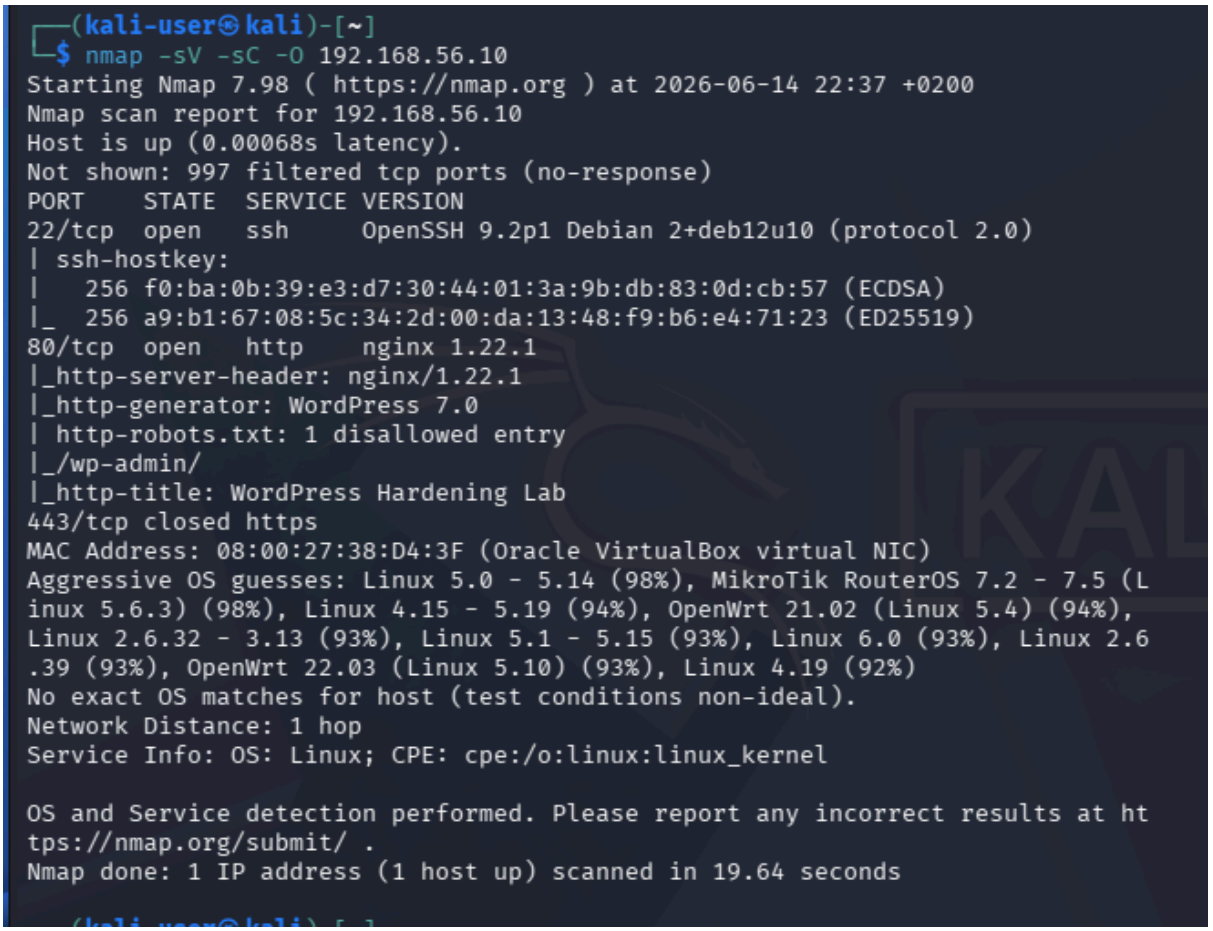
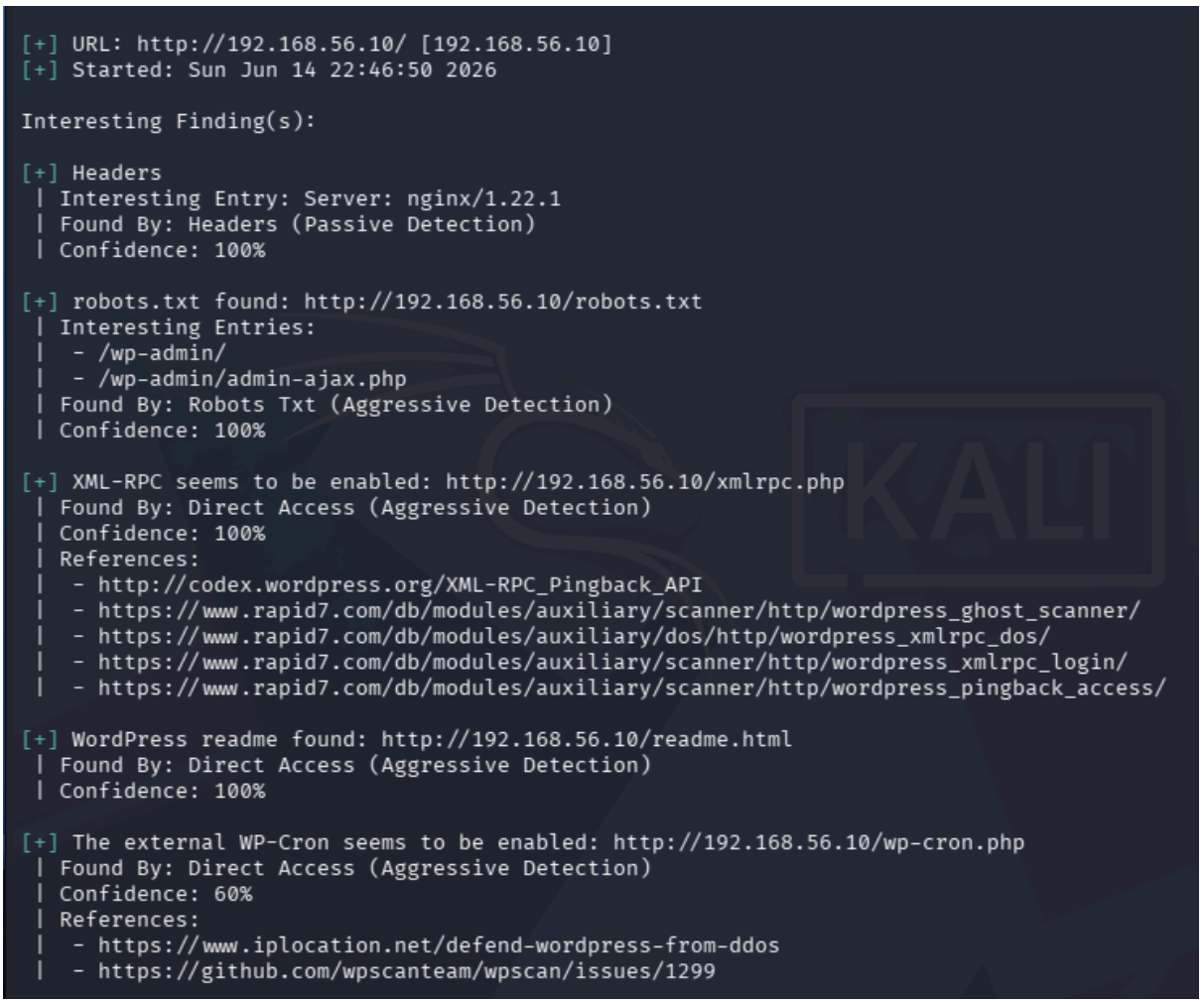


Figure 14: Output nmap scan from kali-user before hardening

It is not safe that Nginx and WordPress show their versions, that the WordPress admin folder is visible and that you can see the title. We want to change that through hardening.

wpscan --url http://192.168.56.10 --enumerate vp,vt,u



```
[+] URL: http://192.168.56.10/ [192.168.56.10]
[+] Started: Sun Jun 14 22:46:50 2026

Interesting Finding(s):

[+] Headers
 | Interesting Entry: Server: nginx/1.22.1
 | Found By: Headers (Passive Detection)
 | Confidence: 100%

[+] robots.txt found: http://192.168.56.10/robots.txt
 | Interesting Entries:
 |  - /wp-admin/
 |  - /wp-admin/admin-ajax.php
 | Found By: Robots Txt (Aggressive Detection)
 | Confidence: 100%

[+] XML-RPC seems to be enabled: http://192.168.56.10/xmlrpc.php
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 100%
 | References:
 |  - http://codex.wordpress.org/XML-RPC_Pingback_API
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghost_scanner/
 |  - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_pingback_access/

[+] WordPress readme found: http://192.168.56.10/readme.html
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 100%

[+] The external WP-Cron seems to be enabled: http://192.168.56.10/wp-cron.php
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 60%
 | References:
 |  - https://www.iplocation.net/defend-wordpress-from-ddos
 |  - https://github.com/wpscanteam/wpscan/issues/1299
```

Figure 15: Output wpscan 1 before hardening

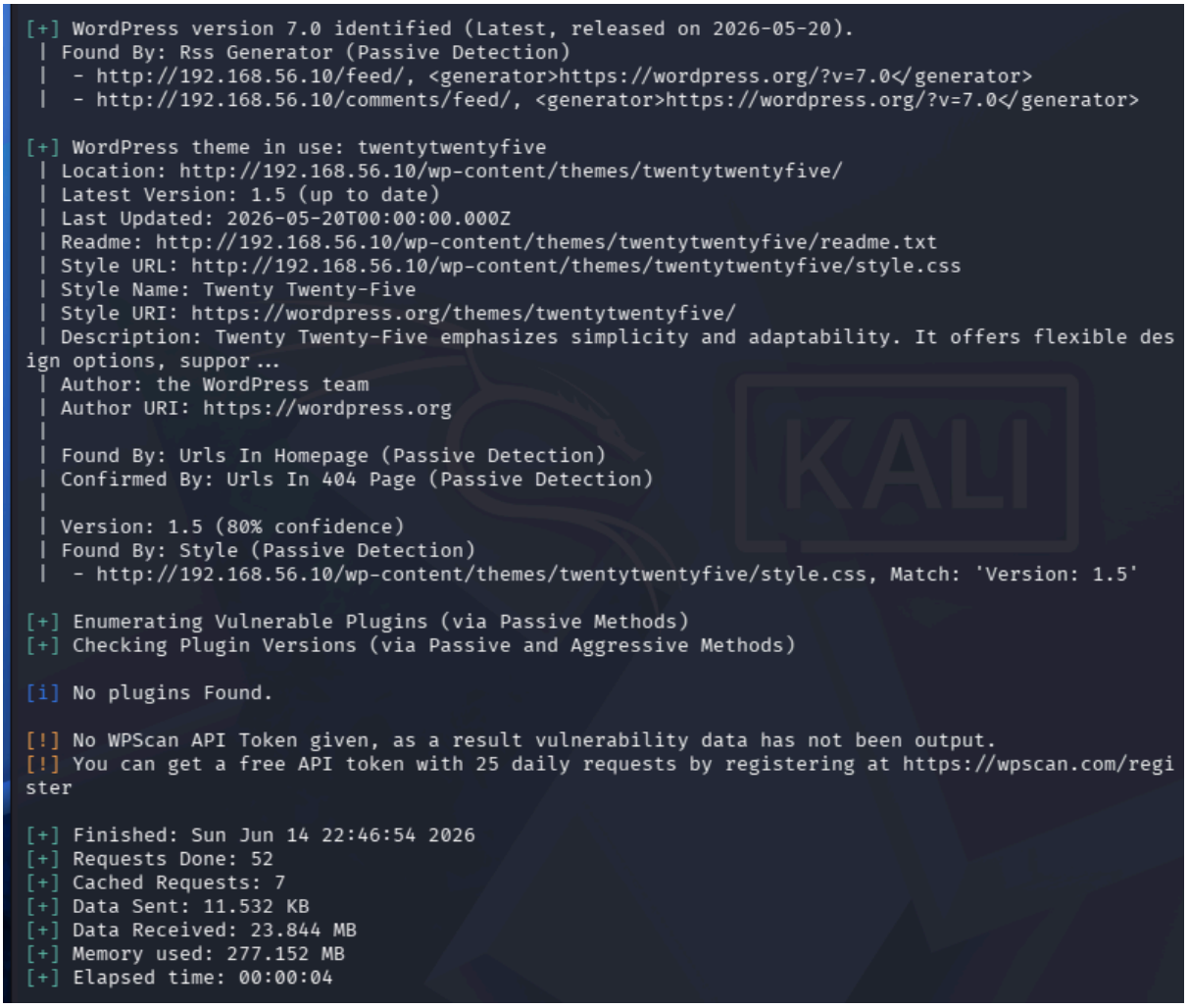


Figure 16: Output wpscan 2 before hardening

This also shows the versions of nginx, WordPress and the theme. Additionally, WordPress readme gives a lot of information, wp-cron.php is accessible, we can see the admin path again and XML-RPC is enabled. We want to change all of that.

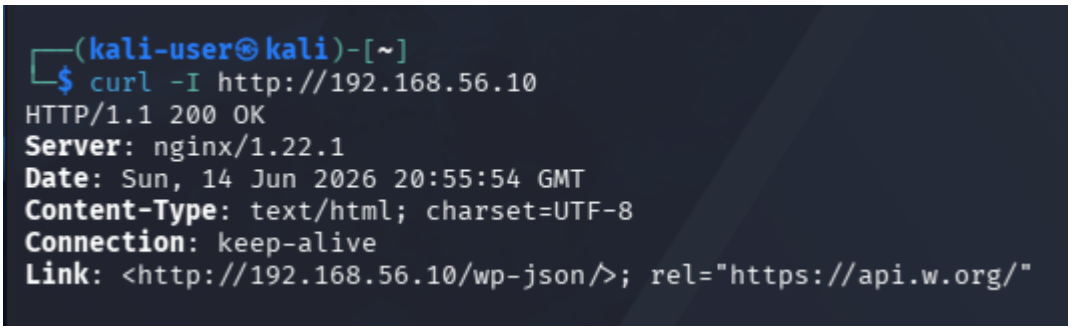


Figure 17: Output curl from kali-user before hardening

API and Nginx versions are visible. There is no HTTP/2 and so security headers. We also want to change that.

While checking the wordpress rights, we can see that wp-config.php is owned by root:root. We want to change that to wp-user:www-data.

```
wp-viki@wp-server:~$ ls -la /var/www/wordpress
total 248
drwxr-xr-x  5 wp-user www-data  4096 Jun 14 21:30 .
drwxr-xr-x  4 root    root      4096 Jun 14 20:44 ..
-rw-r--r--  1 wp-user www-data   405 Jun 14 20:44 index.php
-rw-r--r--  1 wp-user www-data 19903 Jun 14 20:44 license.txt
-rw-r--r--  1 wp-user www-data  7406 Jun 14 20:44 readme.html
-rw-r--r--  1 wp-user www-data  7371 Jun 14 20:44 wp-activate.php
drwxr-xr-x  9 wp-user www-data  4096 Jun 14 20:44 wp-admin
-rw-r--r--  1 wp-user www-data   351 Jun 14 20:44 wp-blog-header.php
-rw-r--r--  1 wp-user www-data  2323 Jun 14 20:44 wp-comments-post.php
-rw-r--r--  1 root    root      3616 Jun 14 21:30 wp-config.php
-rw-r--r--  1 wp-user www-data  3339 Jun 14 20:44 wp-config-sample.php
drwxr-xr-x  4 wp-user www-data  4096 Jun 14 20:44 wp-content
-rw-r--r--  1 wp-user www-data  5617 Jun 14 20:44 wp-cron.php
drwxr-xr-x 35 wp-user www-data 16384 Jun 14 20:44 wp-includes
-rw-r--r--  1 wp-user www-data  2493 Jun 14 20:44 wp-links-opml.php
-rw-r--r--  1 wp-user www-data  3937 Jun 14 20:44 wp-load.php
-rw-r--r--  1 wp-user www-data 51850 Jun 14 20:44 wp-login.php
-rw-r--r--  1 wp-user www-data  8727 Jun 14 20:44 wp-mail.php
-rw-r--r--  1 wp-user www-data 32650 Jun 14 20:44 wp-settings.php
-rw-r--r--  1 wp-user www-data 34621 Jun 14 20:44 wp-signup.php
-rw-r--r--  1 wp-user www-data  5214 Jun 14 20:44 wp-trackback.php
-rw-r--r--  1 wp-user www-data  3205 Jun 14 20:44 xmlrpc.php
```

Figure 18: Output ls -la

sudo chown -R wp-user:www-data /var/www/wordpress - sets wp-user as owner and www-data as group for all files in that directory
sudo find /var/www/wordpress -type d -exec chmod 755 {} \; - allows read (r), write (w), execute (x) for directories by the owner, r, x by the group and r, x by others (you need x to access files)
sudo find /var/www/wordpress -type f -exec chmod 644 {} \; - allows r, w for files by the owner, r by the group and r for others (Nginx / PHP can read WordPress files but can not change them)
sudo chmod 640 /var/www/wordpress/wp-config.php - allows r, w for wp-config.php by the owner, r by the group and nothing by others (others should not be able to read database passwords)

We check if there are world-writable files or directories.

```
wp-viki@wp-server:~$ find /var/www/wordpress -type d -perm -002 -ls
wp-viki@wp-server:~$ find /var/www/wordpress -type f -perm -002 -ls
wp-viki@wp-server:~$
```

Figure 19: Output find world-writable files or directories

Nothing is found. That means all rights are set correctly.

WordPress hardening

Now we start the hardening part.

Now we add to the wp-config.php:
define('DISALLOW_FILE_EDIT', true); - deactivates theme / plugin editor to not be able to change php files via the admin panel
define('DISABLE_WP_CRON', true); - we want to use a real cron, wp-cron.php could be accessed easily from outside

To create a real cron:
sudo crontab -u wp-user -e and choose nano as editor
and add:
*/5 * * * * /usr/bin/php /var/www/wordpress/wp-cron.php > /dev/null 2>&1 - execute wp-cron.php every 5 minutes by system cron

To install wp-cli, first we have to install curl because it is not included in the minimal Debian installation. Then we use curl to install wp-cli. Doing everything through the web dashboard would make the setup vulnerable. Also because of wp-cli we can use cron jobs or deploy scripts for updates. That's why we want wp-cli. To install it for the whole system, we then use:
sudo chmod +x wp-cli.phar
sudo mv wp-cli.phar /usr/local/bin/wp

We look at plugins and themes next.

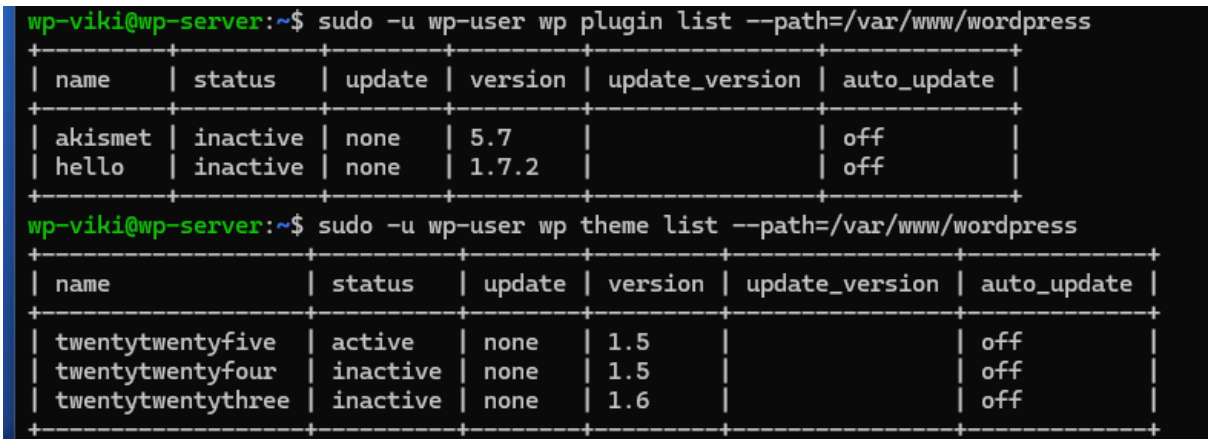


Figure 20: Output existing plugins and themes

Inactive themes and plugins add unnecessary risks. Even if we don't use them, they can be attacked and might have vulnerabilities. That's why we delete them.
For the active theme we don't activate auto updates because they come from third parties and might bring some problems. OS updates and WordPress core security updates are well tested before being launched. For plugins and themes that might not be the case. In addition to that, they might include new dependencies, need a php version that we don't have or even include malware. That's why it is safer to update manually after and update.

Now fail2ban.

```
wp-viki@wp-server:~$ sudo fail2ban-client status
Status
|- Number of jail:      1
`- Jail list:   sshd
wp-viki@wp-server:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     0
   `- Banned IP list:
wp-viki@wp-server:~$ sudo ls /etc/fail2ban/jail.d/
defaults-debian.conf
```

Figure 21: Fail2ban status 1

```
wp-viki@wp-server:~$ sudo cat /etc/fail2ban/jail.d/defaults-debian.conf
[sshd]
enabled = true
```

Figure 22: Fail2ban status 2

We can see that 1 jail is active:sshd and we don't use specific WordPress or Nginx jails. To keep the login safe we will later do that through Nginx rate limiting. The rest looks good for now.

Next we are looking at Nginx.

```
wp-viki@wp-server:~$ sudo ls /etc/nginx/sites-enabled/
wordpress
wp-viki@wp-server:~$ sudo cat /etc/nginx/sites-enabled/default 2>/dev/null || sudo cat /etc/nginx/sites-enabled/*
server {
    listen 80;
    server_name _;
    root /var/www/wordpress;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.2-fpm.sock;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

Figure 23: Nginx enabled sites

Our first setup was minimal just to have a starting point. We want to change the whole configuration because we want HTTPS, security header and path filters. First, we create a self-signed certificate. In a real scenario not use a self-signed certificate because Lets's Encrypt would need to access our server from the outside and we would need a real domain. A self-signed certificate still gives us more security for the traffic (TLS works) and we can try our wanted setup.

Our new config includes:
limit_req_zone $binary_remote_addr zone=wp_login:10m rate=5r/m; - saves ip addresses and limits the max request to 5 per minute, needed in wp-login.php block

```
server {
    listen 80;
    return 301 https://$host$request_uri;
}
```

- every http request is redirected to https, is safer

ssl_protocols TLSv1.2 TLSv1.3; - only newer TLS versions because they are safer
ssl_ciphers HIGH:!aNULL:!MD5; - no weak ciphers
server_tokens off; - Nginx version not visible
add_header Strict-Transport-Security "max-age=31536000" always; - always use https for the next year, no downgrade attacks possible
add_header X-Content-Type-Options "nosniff" always; - no mime sniffing
add_header X-Frame-Options "SAMEORIGIN" always; - no iframe use, no clickjacking
add_header Referrer-Policy "no-referrer-when-downgrade" always; - better control of url information

```
location / {
    try_files $uri $uri/ /index.php$args;
}
```

- standard WordPress routing, if no file exists use index.php

```
location ~ \.php$ {
    include snippets/fastcgi-php.conf;
    fastcgi_pass unix:/run/php/php8.2-fpm.sock;
}
```

- all .php files are sent to php-fpm via unix socket, faster and safer than tcp

```
location = /xmlrpc.php {
    deny all;
}
```

- xml-rpc is an old WordPress api, often used for brute force or ddos attacks, already deactivated in wp-config.php, also block it here for defense in depth

```
location = /wp-login.php {
    limit_req zone=wp_login burst=3 nodelay;
    include snippets/fastcgi-php.conf;
    fastcgi_pass unix:/run/php/php8.2-fpm.sock;
}
```

- executes rate limiting zone, no brute forcing the login form

```
location ~* \.(env|git|sql|bak|log)$ {
    deny all;
}
location = /wp-config.php {
```

```
    deny all;
}
```

- never access sensitive files, including credentials and configuration

```
location ~* /wp-content/uploads/.*\.php$ {
    deny all;
}
```

- no execution of php files from this directory

```
location ~ /\.ht {
    deny all;
}
```

- .htaccess is used by apache, sometimes wordpress creates it automatically, could include sensitive information, also no access to .htpasswd and other files starting with .ht

Let's test the config via the second vm. (-k to ignore the warning of self-signed certificate)

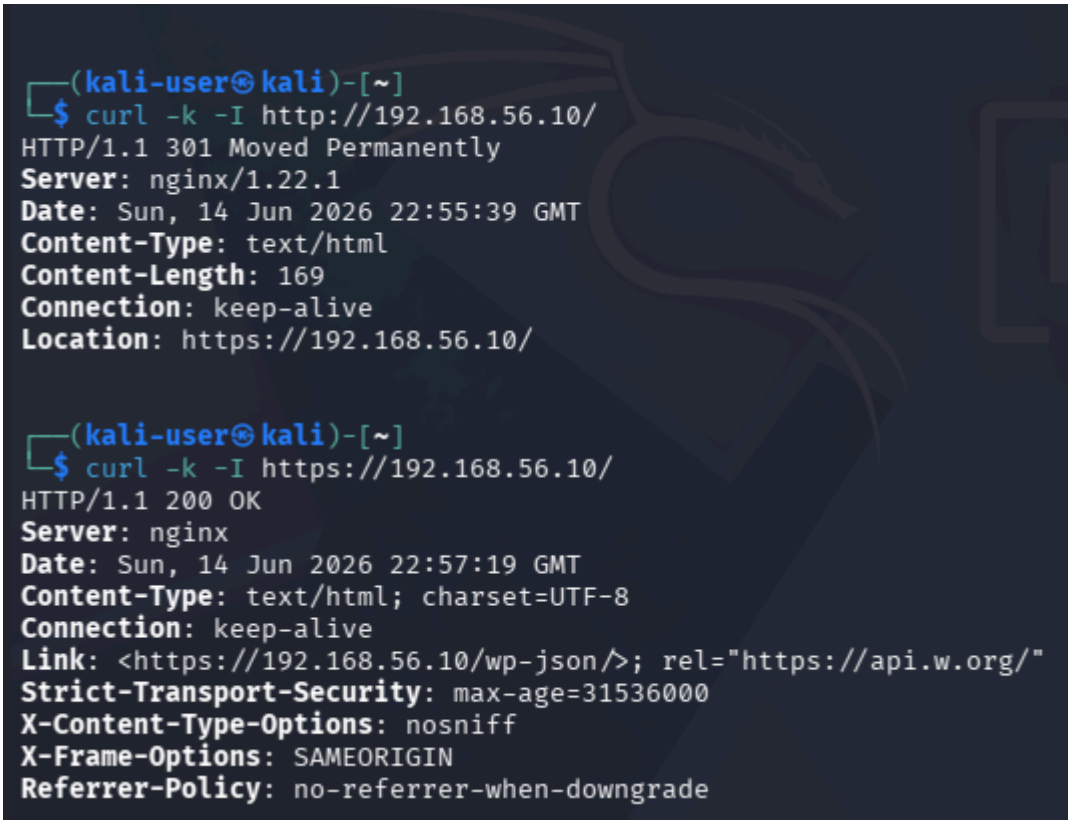


Figure 24: Curl via kali-user to try config 1

http to https redirect works, https works, security header visible, Nginx without version
That's exactly what we wanted.

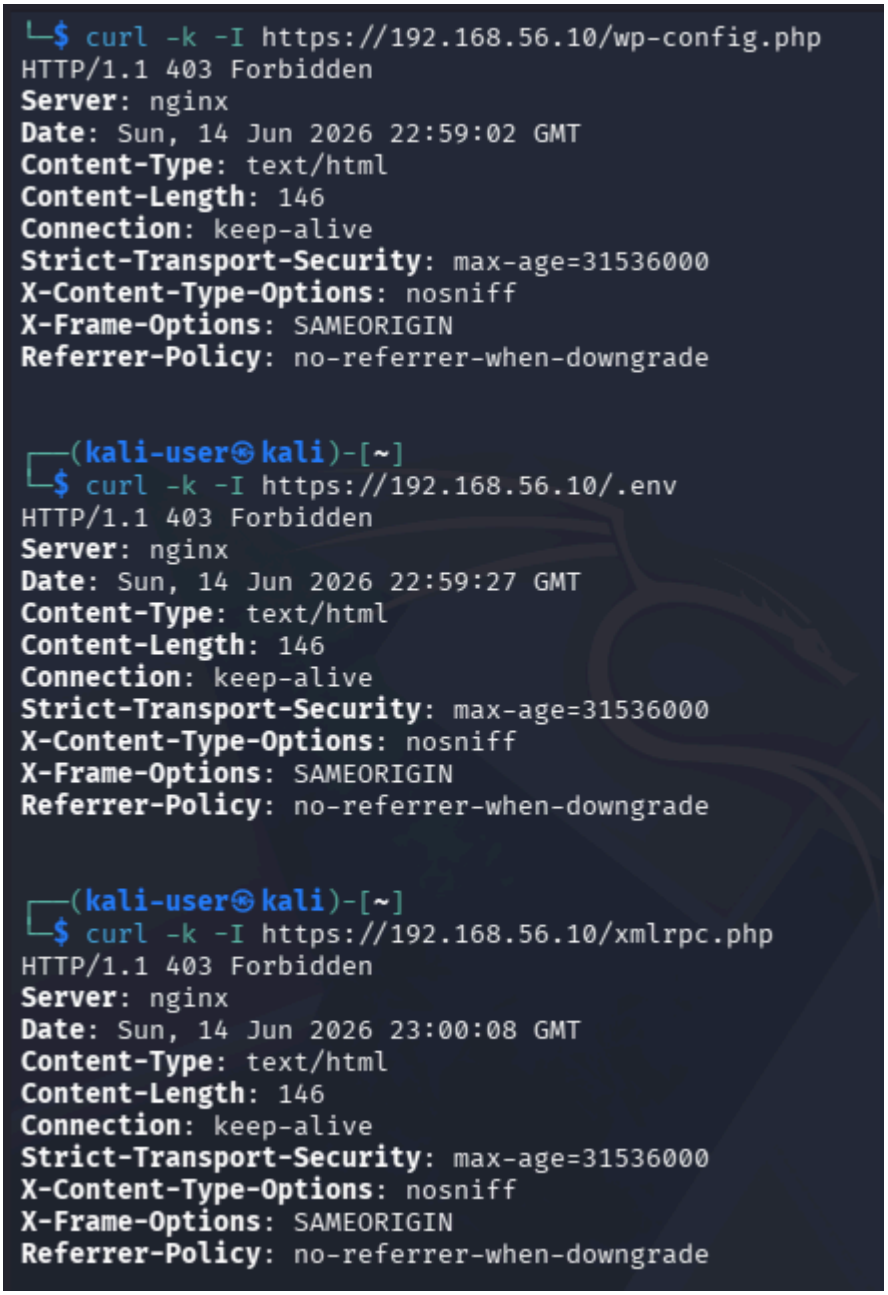


Figure 25: Curl via kali-user to try config 2

all tested sensitive paths are forbidden, security header visible for forbidden response too
Perfect.
In wp-server:
echo '<?php phpinfo(); ?>' | sudo tee /var/www/wordpress/wp-content/uploads/test.php
Then in the second vm:



Figure 26: Curl via kali-user to try config 3

It should be forbidden too. Trying the same code without -I showed a lot of text. It looks like the php was executed, which should not happen. Let's fix our config. We change the php block to:

```
location ~ \.php$ {
    if ($request_uri ~* "/wp-content/uploads/.*\.php$") {
        return 403;
    }
    include snippets/fastcgi-php.conf;
    fastcgi_pass unix:/run/php/php8.2-fpm.sock;
}
```

- to block php in uploads

Now we try again in the second vm.



Figure 27: Curl via kali-user to try config 4

Good, now php is also blocked.
Now we realise that http/1.1 is used but we wanted http2. So again in the config file we change listen 443 ssl to listen 443 ssl http2. Http/2 is faster, more efficient and uses TLS. Http/ would even be better but the setup is hard with Debian 12. Also it is not used so much yet. So we choose http/2.

```
┌──(kali-user㉿kali)-[~]
└─$ curl -k -I https://192.168.56.10/
HTTP/2 200
server: nginx
date: Sun, 14 Jun 2026 23:24:05 GMT
content-type: text/html; charset=UTF-8
link: <https://192.168.56.10/wp-json/>; rel="https://api.w.org/"
strict-transport-security: max-age=31536000
x-content-type-options: nosniff
x-frame-options: SAMEORIGIN
referrer-policy: no-referrer-when-downgrade
```

Figure 28: Curl via kali-user to try config 5

Now that works too.

PHP-FPM, MariaDB, AppArmor, auditd, Backup

To harden php-fpm we check some settings.

```
wp-viki@wp-server:~$ sudo cat /etc/php/8.2/fpm/php.ini | grep -E 'expose_php|display_errors|allow_url_fopen|allow_url_in
clude|max_execution_time|memory_limit|upload_max_filesize|post_max_size'
; display_errors
expose_php = Off
max_execution_time = 30
memory_limit = 128M
display_errors = Off
; separately from display_errors. We strongly recommend you set this to 'off'
post_max_size = 8M
upload_max_filesize = 2M
allow_url_fopen = On
allow_url_include = Off
```

Figure 29: Output current settings in php.ini

We can see, that the php version is not visible, php errors will not be shown in the browser, including urls is not possible, scripts have a max time so that broken scripts wont block the server, memory limit is active against memory exhaustion attacks, limit for filesize and post request size are active too.
The only thing we still want to change is to access urls from the outside.
sudo nano /etc/php/8.2/fpm/php.ini
change allow_url_fopen = On to = Off

Our next step would normally be hardening mariadb. Since we used the secure installation, we will just check if the settings are like we want them to be.

```
wp-viki@wp-server:~$ sudo grep -r 'bind-address' /etc/mysql/
/etc/mysql/mariadb.conf.d/50-server.cnf:bind-address            = 127.0.0.1
/etc/mysql/mariadb.conf.d/60-galera.cnf:#bind-address = 0.0.0.0
wp-viki@wp-server:~$ sudo mariadb -e "SHOW GRANTS FOR 'wp-db-user'@'localhost';"
+-----------------------------------------------------------------------------------------------------------------+
| Grants for wp-db-user@localhost                                                                                 |
+-----------------------------------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `wp-db-user`@`localhost` IDENTIFIED BY PASSWORD [redacted]                                |
| GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, INDEX, ALTER ON `wordpress`.* TO `wp-db-user`@`localhost`   |
+-----------------------------------------------------------------------------------------------------------------+
```

Figure 30: Output to verify MariaDB hardening 1

MariaDB is only accessible via localhost like we wanted and the rights for wp-db-user are also to our liking.

```
wp-viki@wp-server:~$ sudo mariadb -e "SELECT user, host, plugin FROM mysql.user;"
+-------------+-----------+-----------------------+
| User        | Host      | plugin                |
+-------------+-----------+-----------------------+
| mariadb.sys | localhost | mysql_native_password |
| root        | localhost | mysql_native_password |
| mysql       | localhost | mysql_native_password |
| wp-db-user  | localhost | mysql_native_password |
+-------------+-----------+-----------------------+
wp-viki@wp-server:~$ sudo mariadb -e "SHOW DATABASES;"
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| wordpress          |
+--------------------+
```

Figure 31: Output to verify MariaDB hardening 2

This shows us that there is no anonymous user, root only via local host, no remote root, only wp-db-user as user. There is no test database, the system databases are here and our additional wordpress database too. We are happy with this.

Now we look at the wp-cli update process.

```
wp-viki@wp-server:~$ sudo -u wp-user wp core version --path=/var/www/wordpress
7.0
wp-viki@wp-server:~$ sudo -u wp-user wp core check-update --path=/var/www/wordpress
Success: WordPress is at the latest version.
wp-viki@wp-server:~$ sudo -u wp-user wp plugin list --path=/var/www/wordpress
+------+--------+--------+---------+----------------+-------------+
| name | status | update | version | update_version | auto_update |
+------+--------+--------+---------+----------------+-------------+
+------+--------+--------+---------+----------------+-------------+
wp-viki@wp-server:~$
```

Figure 32: Output wp-cli update process

WordPress has the newest version already. Since we don’t have any plugins, we don’t need to update them. Otherwise we would run wp core update and wp plugin update.

Next we look at resource limits.

```
wp-viki@wp-server:~$ sudo cat /etc/php/8.2/fpm/pool.d/www.conf | grep -E 'pm\.|pm =|request_terminate_timeout'
listen = /run/php/php8.2-fpm.sock
;   static  - a fixed number (pm.max_children) of child processes;
;             pm.max_children      - the maximum number of children that can
;             pm.start_servers     - the number of children created on startup.
;             pm.min_spare_servers - the minimum number of children in 'idle'
;             pm.max_spare_servers - the maximum number of children in 'idle'
;             pm.max_spawn_rate    - the maximum number of rate to spawn child
;             pm.max_children           - the maximum number of children that
;             pm.process_idle_timeout   - The number of seconds after which
pm = dynamic
; forget to tweak pm.* to fit your needs.
pm.max_children = 5
pm.start_servers = 2
pm.min_spare_servers = 1
pm.max_spare_servers = 3
;pm.max_spawn_rate = 32
;pm.process_idle_timeout = 10s;
;pm.max_requests = 500
;pm.status_path = /status
;pm.status_listen = 127.0.0.1:9001
;request_terminate_timeout = 0
; The timeout set by 'request_terminate_timeout' ini option is not engaged after
;request_terminate_timeout_track_finished = no
```

Figure 33: Output current resource limits

pm.max_children = 5 looks okay for our use case
pm.max_requests = 500 is not active, we will change that, protects against memory leaks
request_terminate_timeout = 0 means broken scripts could run for forever, let's change that to 30s. That should be more than enough time for working scripts.

In production we would track and then limit accordingly. (ps aux | php-fpm) Maybe max_children would then be around 30 or more. We would also check our logs and monitoring to see how many requests normally arrive at the same time, how long a request takes in general and if there are any peaks. With ab (Apache Benchmark) or k6 we could simulate real load to test the limits. Normally limits would be set carefully and changed again if necessary. Never go to max by default. Also set alarms to be able to react in time. Memory_test.c and process_test.c could also be used to test systemd/limits.conf in production.

```
wp-viki@wp-server:~$ sudo systemctl status php8.2-fpm
● php8.2-fpm.service - The PHP 8.2 FastCGI Process Manager
     Loaded: loaded (/lib/systemd/system/php8.2-fpm.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-06-14 20:23:06 CEST; 5h 44min ago
       Docs: man:php-fpm8.2(8)
    Process: 120583 ExecReload=/bin/kill -USR2 $MAINPID (code=exited, status=0/SUCCESS)
   Main PID: 44992 (php-fpm8.2)
     Status: "Ready to handle connections"
      Tasks: 3 (limit: 4653)
     Memory: 14.9M
        CPU: 5.250s
     CGroup: /system.slice/php8.2-fpm.service
             ├─ 44992 "php-fpm: master process (/etc/php/8.2/fpm/php-fpm.conf)"
             ├─120584 "php-fpm: pool www"
             └─120585 "php-fpm: pool www"

Jun 14 20:23:06 wp-server systemd[1]: Starting php8.2-fpm.service - The PHP 8.2 FastCGI Process Manager...
Jun 14 20:23:06 wp-server systemd[1]: Started php8.2-fpm.service - The PHP 8.2 FastCGI Process Manager.
Jun 15 01:38:09 wp-server systemd[1]: Reloading php8.2-fpm.service - The PHP 8.2 FastCGI Process Manager...
Jun 15 01:38:09 wp-server systemd[1]: Reloaded php8.2-fpm.service - The PHP 8.2 FastCGI Process Manager.
Jun 15 01:38:10 wp-server php-fpm8.2[44992]: [15-Jun-2026 01:38:10] NOTICE: using inherited socket fd=8, "/run/php/php8>
Jun 15 02:07:29 wp-server systemd[1]: Reloading php8.2-fpm.service - The PHP 8.2 FastCGI Process Manager...
Jun 15 02:07:29 wp-server systemd[1]: Reloaded php8.2-fpm.service - The PHP 8.2 FastCGI Process Manager.
Jun 15 02:07:29 wp-server php-fpm8.2[44992]: [15-Jun-2026 02:07:29] NOTICE: using inherited socket fd=8, "/run/php/php8>
```

Figure 34: Output systemctl status for php-fpm

php-fpm runs without problems, ready to handle connections, no errors in logs

AppArmor is our next step.

```
wp-viki@wp-server:~$ sudo apparmor_status
apparmor module is loaded.
10 profiles are loaded.
10 profiles are in enforce mode.
   /usr/bin/man
   /usr/lib/NetworkManager/nm-dhcp-client.action
   /usr/lib/NetworkManager/nm-dhcp-helper
   /usr/lib/connman/scripts/dhclient-script
   /{,usr/}sbin/dhclient
   lsb_release
   man_filter
   man_groff
   nvidia_modprobe
   nvidia_modprobe//kmod
0 profiles are in complain mode.
0 profiles are in kill mode.
0 profiles are in unconfined mode.
1 processes have profiles defined.
1 processes are in enforce mode.
   /usr/sbin/dhclient (33069) /{,usr/}sbin/dhclient
0 processes are in complain mode.
0 processes are unconfined but have a profile defined.
0 processes are in mixed mode.
0 processes are in kill mode.
```

Figure 35: Output current AppArmor status

There we see that the module is loaded, there are 10 profiles in enforce mode (all system related so that's okay). But there are no profiles for Nginx, php-fpm and mariadb.
We install apparmor-profiles and apparmor-profiles-extra to check if there are profiles for those available.

```
wp-viki@wp-server:~$ sudo apparmor_status | grep -E 'nginx|php|mysql|mariadb'
   php-fpm
```

Figure 36: Output AppArmor status for Nginx, PHP, MySQL and MariaDB 1

There is only one for php-fpm. We activate this one. To be able to use aa-enforce to activate it, we need to install apparmor-utils first.

```
wp-viki@wp-server:~$ sudo aa-enforce /etc/apparmor.d/php-fpm
Setting /etc/apparmor.d/php-fpm to enforce mode.
wp-viki@wp-server:~$ sudo apparmor_status | grep -E 'nginx|php|mysql|mariadb'
   php-fpm
```

Figure 37: Output AppArmor status for Nginx, PHP, MySQL and MariaDB 2

```
wp-viki@wp-server:~$ sudo systemctl status php8.2-fpm
● php8.2-fpm.service - The PHP 8.2 FastCGI Process Manager
     Loaded: loaded (/lib/systemd/system/php8.2-fpm.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-06-14 20:23:06 CEST; 6h ago
       Docs: man:php-fpm8.2(8)
    Process: 120583 ExecReload=/bin/kill -USR2 $MAINPID (code=exited, status=0/SUCCESS)
   Main PID: 44992 (php-fpm8.2)
     Status: "Processes active: 0, idle: 2, Requests: 0, slow: 0, Traffic: 0.00req/sec"
      Tasks: 3 (limit: 4653)
     Memory: 14.9M
        CPU: 5.387s
     CGroup: /system.slice/php8.2-fpm.service
             ├─ 44992 "php-fpm: master process (/etc/php/8.2/fpm/php-fpm.conf)"
             ├─120584 "php-fpm: pool www"
             └─120585 "php-fpm: pool www"
```

Figure 38: Output to test systemctl status of php-fpm again

php-fpm is still running without problems.

```
┌──(kali-user㉿kali)-[~]
└─$ curl -k -I https://192.168.56.10/
HTTP/2 200
server: nginx
date: Mon, 15 Jun 2026 00:32:04 GMT
content-type: text/html; charset=UTF-8
link: <https://192.168.56.10/wp-json/>; rel="https://api.w.org/"
strict-transport-security: max-age=31536000
x-content-type-options: nosniff
x-frame-options: SAMEORIGIN
referrer-policy: no-referrer-when-downgrade
```

Figure 39: Curl via kali-user to test current settings

Everything is the same as before so AppArmore doesn't block anything.
In production we would also write profiles for Nginx and MariaDB but for our lab that would be a bit over the top. For us that is a known vulnerability that we will accept for now.

For logging we install auditd and audisp-plugins and enable auditd. We want to be able to see manipulation attempts, have logs so that we can check what happened after an attack, to protect config files. Normal logs could be compromised, changed or deleted by users with root rights. Since auditd runs on the kernel, that is way harder to do.

```
wp-viki@wp-server:~$ sudo systemctl enable --now auditd
Synchronizing state of auditd.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable auditd
wp-viki@wp-server:~$ sudo systemctl status auditd
● auditd.service - Security Auditing Service
     Loaded: loaded (/lib/systemd/system/auditd.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-06-15 02:35:25 CEST; 1min 48s ago
       Docs: man:auditd(8)
             https://github.com/linux-audit/audit-documentation
   Main PID: 121239 (auditd)
      Tasks: 2 (limit: 4653)
     Memory: 688.0K
        CPU: 53ms
     CGroup: /system.slice/auditd.service
             └─121239 /sbin/auditd
```

Figure 40: Output systemctl status of auditd

We activate three rules and test them:

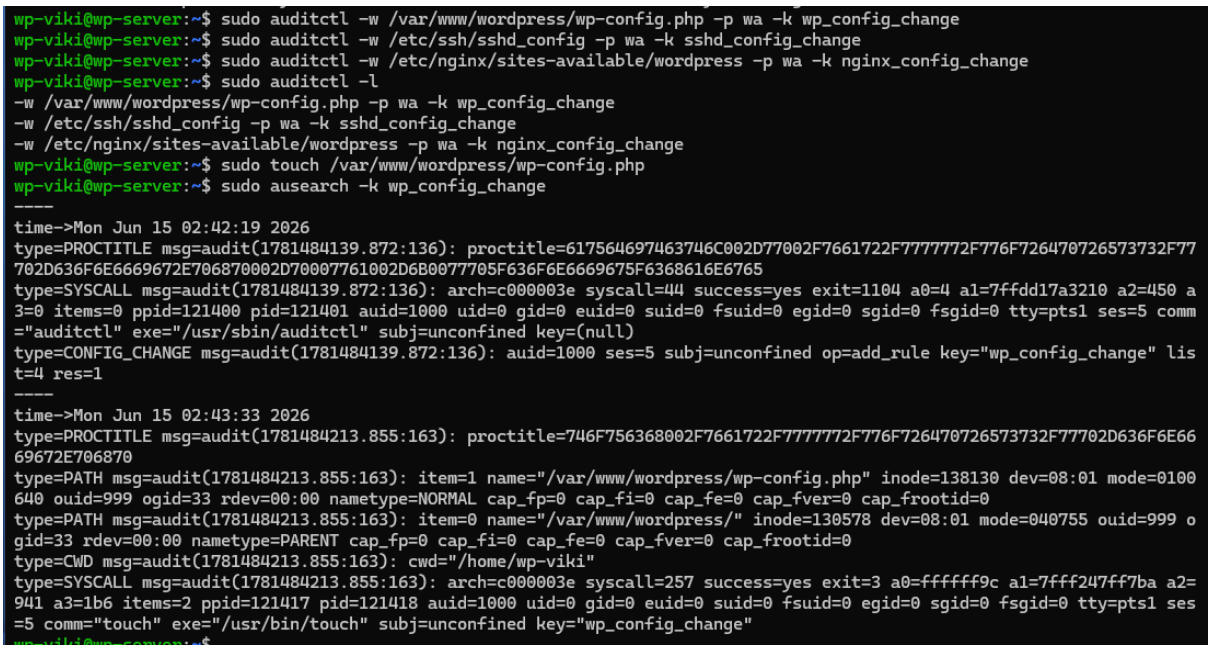


Figure 41: Activate auditd rules and testing them

The first one watches over wp-config.php because database passwords, security keys and critical WordPress configurations are stored here.
The second one watched over the ssh configuration.
The third one watches over Nginx configuration.
We used -p wa because write and attribute change are most important for security reasons. r (read) would be to much because that would create a lot of unnecessary log entries per hour.
The logs show the correct time, which process is used, which file, from which directory and our label. That looks sufficient for us.
To make those rules persistent, we add them to sudo nano /etc/audit/rules.d/wordpress.rules.

We now come to backups. First we create a directory that only the root can access. Then we export the database in a SQL file. We create a checksum to verify later if something was manipulated. Then we save the wp-content directory (themes, uploads, plugins). We don’t need a backup for the core files because we can just download them again. We also create a checksum here. Then we check if both are successfully created.

```
wp-viki@wp-server:~$ sudo mkdir -p /secure-backup
wp-viki@wp-server:~$ sudo chmod 700 /secure-backup
wp-viki@wp-server:~$ sudo mariadb-dump --single-transaction wordpress > /secure-backup/wp.sql
-bash: /secure-backup/wp.sql: Permission denied
wp-viki@wp-server:~$ sudo mariadb-dump --single-transaction wordpress | sudo tee /secure-backup/wp.sql > /dev/null
wp-viki@wp-server:~$ sudo sha256sum /secure-backup/wp.sql
794b28ee7f1dd0b8c5f473714ea1c1b9a4f87cefa21e615002bf05a9ef3cbeb6  /secure-backup/wp.sql
wp-viki@wp-server:~$ sudo tar -czf /secure-backup/wp-content.tar.gz /var/www/wordpress/wp-content
tar: Removing leading `/' from member names
wp-viki@wp-server:~$ sudo sha256sum /secure-backup/wp-content.tar.gz
e57a138f0986ee300359ce6e8348a06e30c97bb3351728ec6d94f8468761daed  /secure-backup/wp-content.tar.gz
wp-viki@wp-server:~$ sudo ls -lh /secure-backup/
total 7.9M
-rw-r--r-- 1 root root 7.8M Jun 15 02:58 wp-content.tar.gz
-rw-r--r-- 1 root root 111K Jun 15 02:57 wp.sql
```

Figure 42: Creating backup

We need backups to later be able to restore our services after an attack. If the checksums are correct, we know the backups are okay.
To see if the restore works, we create a test database and try it out.

```
wp-viki@wp-server:~$ sudo mariadb -e "CREATE DATABASE wp_test;"
wp-viki@wp-server:~$ sudo mariadb wp_test < /secure-backup/wp.sql
-bash: /secure-backup/wp.sql: Permission denied
wp-viki@wp-server:~$ sudo sh -c 'mariadb wp_test < /secure-backup/wp.sql'
wp-viki@wp-server:~$ sudo mariadb -e "SHOW TABLES FROM wp_test;"
+-----------------------+
| Tables_in_wp_test     |
+-----------------------+
| wp_commentmeta        |
| wp_comments           |
| wp_links              |
| wp_options            |
| wp_postmeta           |
| wp_posts              |
| wp_term_relationships |
| wp_term_taxonomy      |
| wp_termmeta           |
| wp_terms              |
| wp_usermeta           |
| wp_users              |
+-----------------------+
wp-viki@wp-server:~$ sudo mariadb -e "DROP DATABASE wp_test;"
```

Figure 43: Restore test

All tables are there so it works correctly. We now delete the test database again because we don't need it anymore.

After Hardening Tests

Now our hardening part is over and we check if everything worked as expected. So via the second vm we try nmap again.

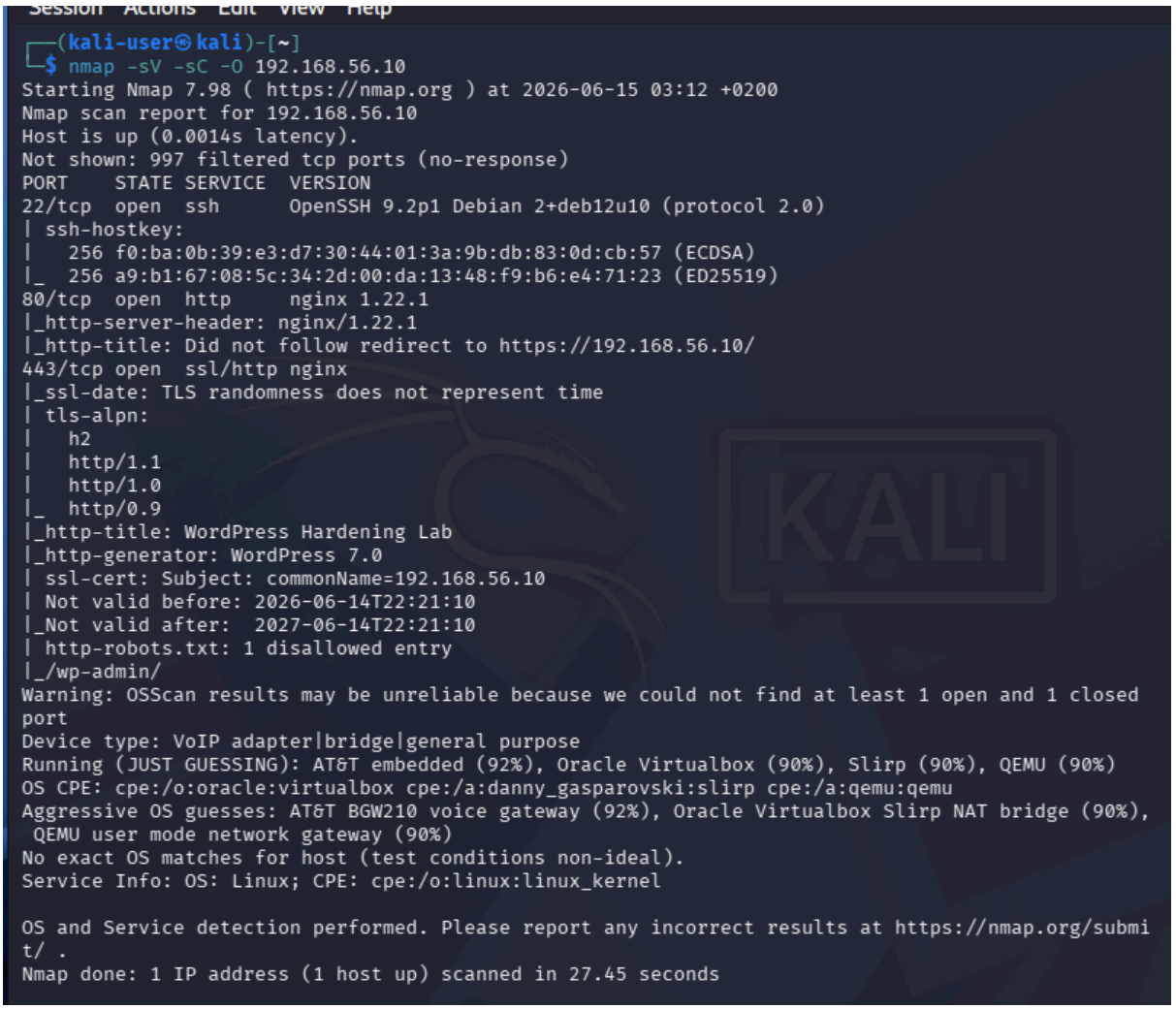


Figure 44: nmap scan via kali-user after hardening 1

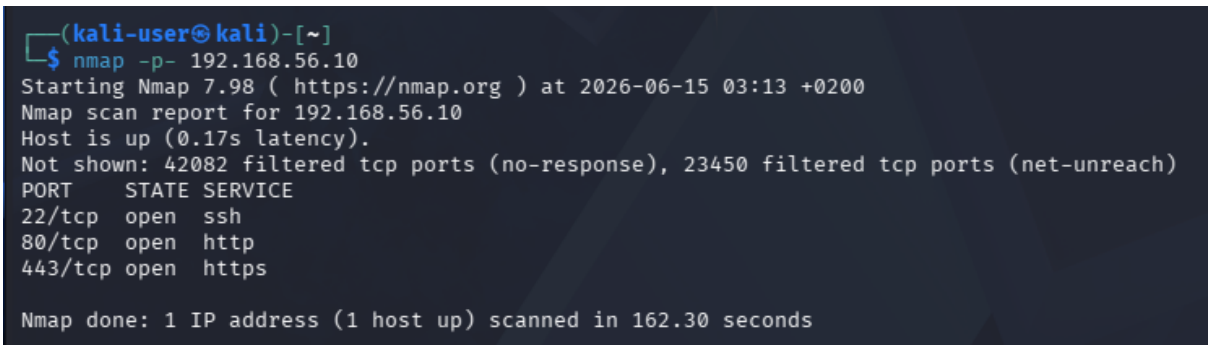


Figure 45: nmap scan via kali-user after hardening 2

We can see that the Nginx Version is still visible. Maybe because we redirect http to https before the https rules would work. We will add server_tokens off there too.
The WordPress version and the admin path are also visible. Let's try to fix that before we continue with the other tests.
So we add server_tokens off; to the http block in /etc/nginx/sites-available/wordpress.
Also in the same file we add:
location /wp-admin/ {
  allow 192.168.56.0/24;

```
    deny all;
}
```

- /wp-admin/ is located in robots.txt. We should not change or delete that but because it is readable from the outside, that would be not safe to keep it that way. So we only allow access for ips from the local network.

At the end of /var/www/wordpress/wp-content/themes/twentytwentyfive/functions.php we add remove_action('wp_head', 'wp_generator');.

```
┌──(kali-user㉿kali)-[~]
└─$ nmap -sV -sC -O 192.168.56.10
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-15 03:33 +0200
Nmap scan report for 192.168.56.10
Host is up (0.0015s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT    STATE SERVICE  VERSION
22/tcp  open  ssh      OpenSSH 9.2p1 Debian 2+deb12u10 (protocol 2.0)
| ssh-hostkey:
|   256 f0:ba:0b:39:e3:d7:30:44:01:3a:9b:db:83:0d:cb:57 (ECDSA)
|_  256 a9:b1:67:08:5c:34:2d:00:da:13:48:f9:b6:e4:71:23 (ED25519)
80/tcp  open  http     nginx
|_http-title: Did not follow redirect to https://192.168.56.10/
443/tcp open  ssl/http nginx
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject: commonName=192.168.56.10
| Not valid before: 2026-06-14T22:21:10
|_Not valid after:  2027-06-14T22:21:10
|_http-title: WordPress Hardening Lab
| tls-alpn:
|   h2
|   http/1.1
|   http/1.0
|_  http/0.9
| http-robots.txt: 1 disallowed entry
|_/wp-admin/
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed
port
Device type: VoIP adapter|bridge|general purpose
Running (JUST GUESSING): AT&T embedded (92%), Oracle Virtualbox (90%), Slirp (90%), QEMU (90%)
OS CPE: cpe:/o:oracle:virtualbox cpe:/a:danny_gasparovski:slirp cpe:/a:qemu:qemu
Aggressive OS guesses: AT&T BGW210 voice gateway (92%), Oracle Virtualbox Slirp NAT bridge (90%),
 QEMU user mode network gateway (90%)
No exact OS matches for host (test conditions non-ideal).
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submi
t/ .
Nmap done: 1 IP address (1 host up) scanned in 27.54 seconds
```

Figure 46: nmap scan via kali-user after hardening + correction

Nginx and WordPress versions are not visible any more. /wp-admin/ is still visible but robots.txt has ip restricted access so that should be even more secure.

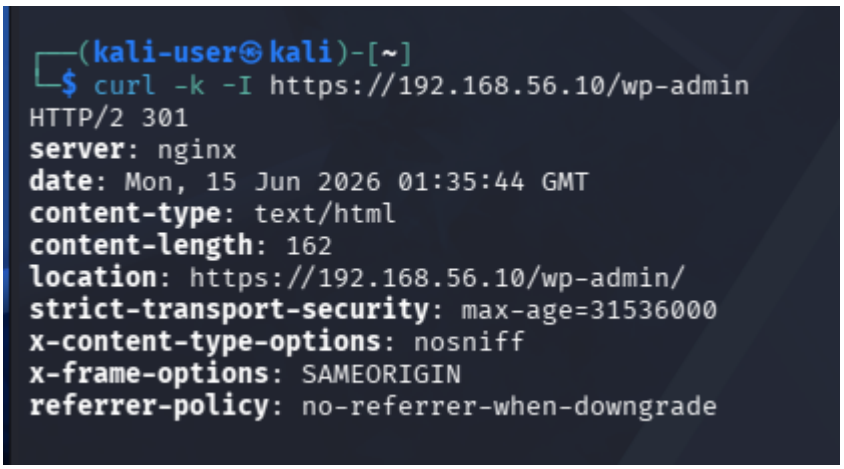


Figure 47: Curl via kali-user after hardening

301: redirect from /wp-admin to /wp-admin/, which is normal for WordPress. Since the second vm is also in the local network, we don't get a 403. We don't have access to a different vm that is not in the local network so we can not try this right now.
Now we try wpscan --url https://192.168.56.10/ --enumerate vp,vt,u --disable-tls-checks. (no tls checks because the scan would cancel because of the self-signed certificate)

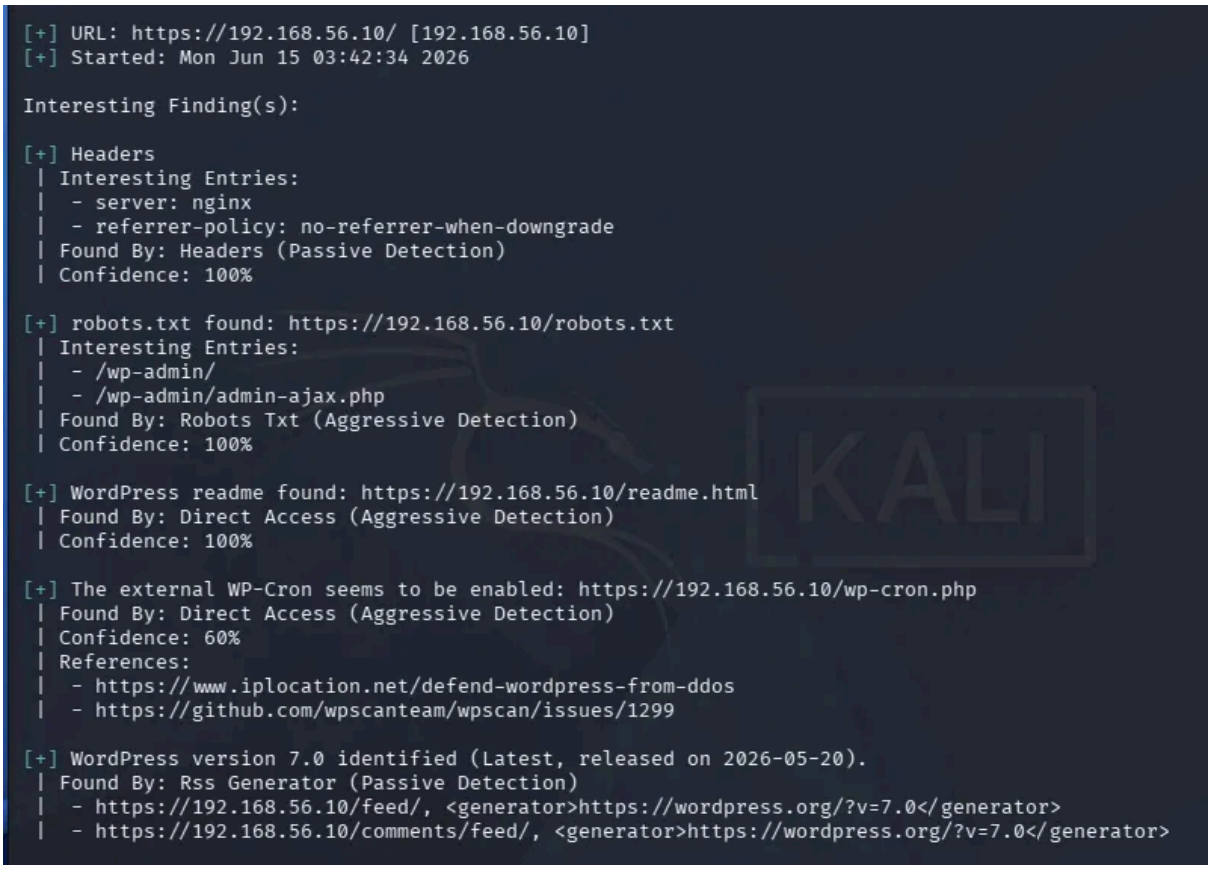


Figure 48: wpscan after hardening 1

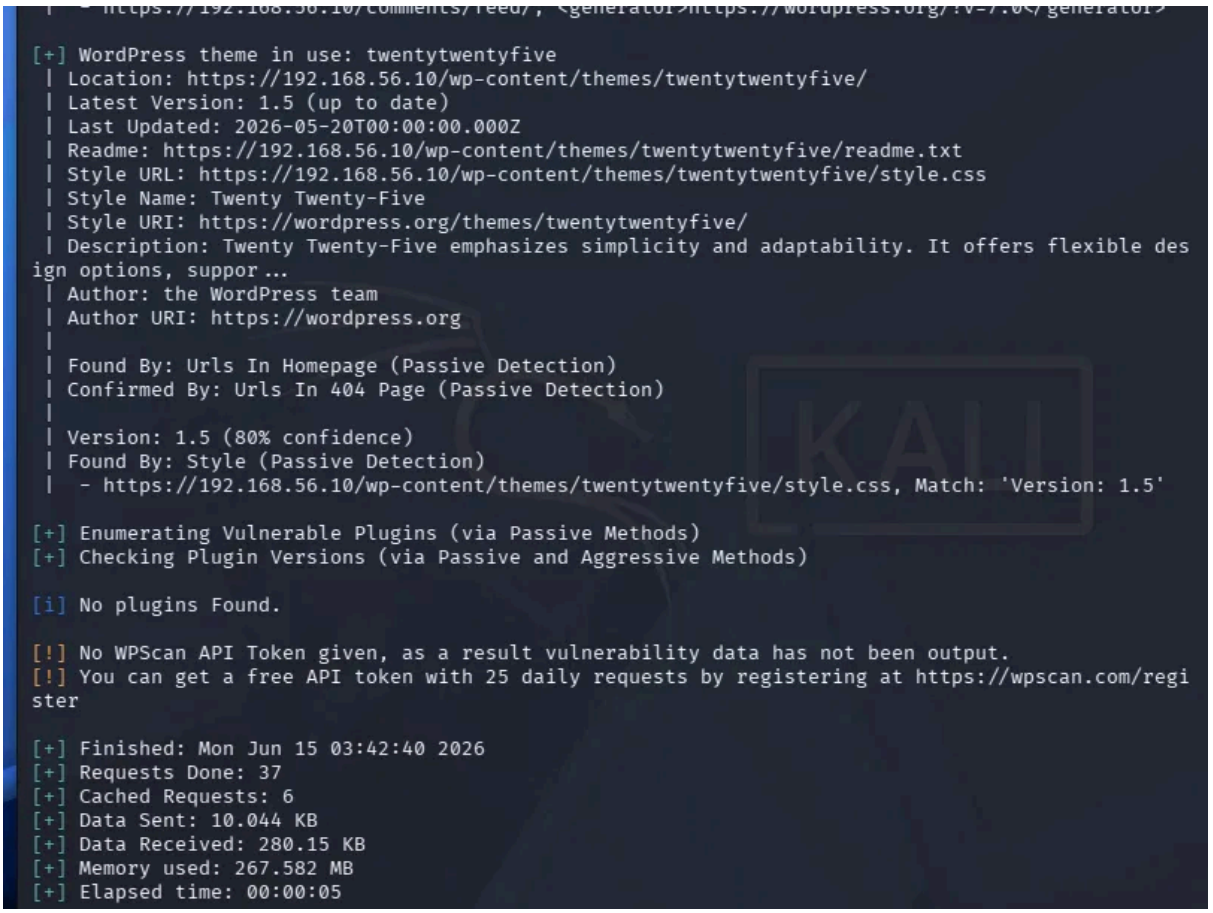


Figure 49: wpscan after hardening 2

Good: no plugins found, theme uptodate, Nginx version not visible
readme.html is still accessible, WordPress version is visible via rss feed and wp_cron should be disabled.
In /etc/nginx/sites-available/wordpress we add:

```
location = /readme.html {
    deny all;
}
```

- denies access for readme

and:

```
location = /wp-cron.php {
    deny all;
}
```

- we already disabled wp_cron in wp-config.php so it is not executed automatically. Still it is possible to access the file via the browser. Now we also block the access from this side. Defense in depth

In /var/www/wordpress/wp-content/themes/twentytwentyfive/functions.php we add:
add_filter('the_generator', '__return_empty_string'); - also not visible via rss feed
Try the scan again.

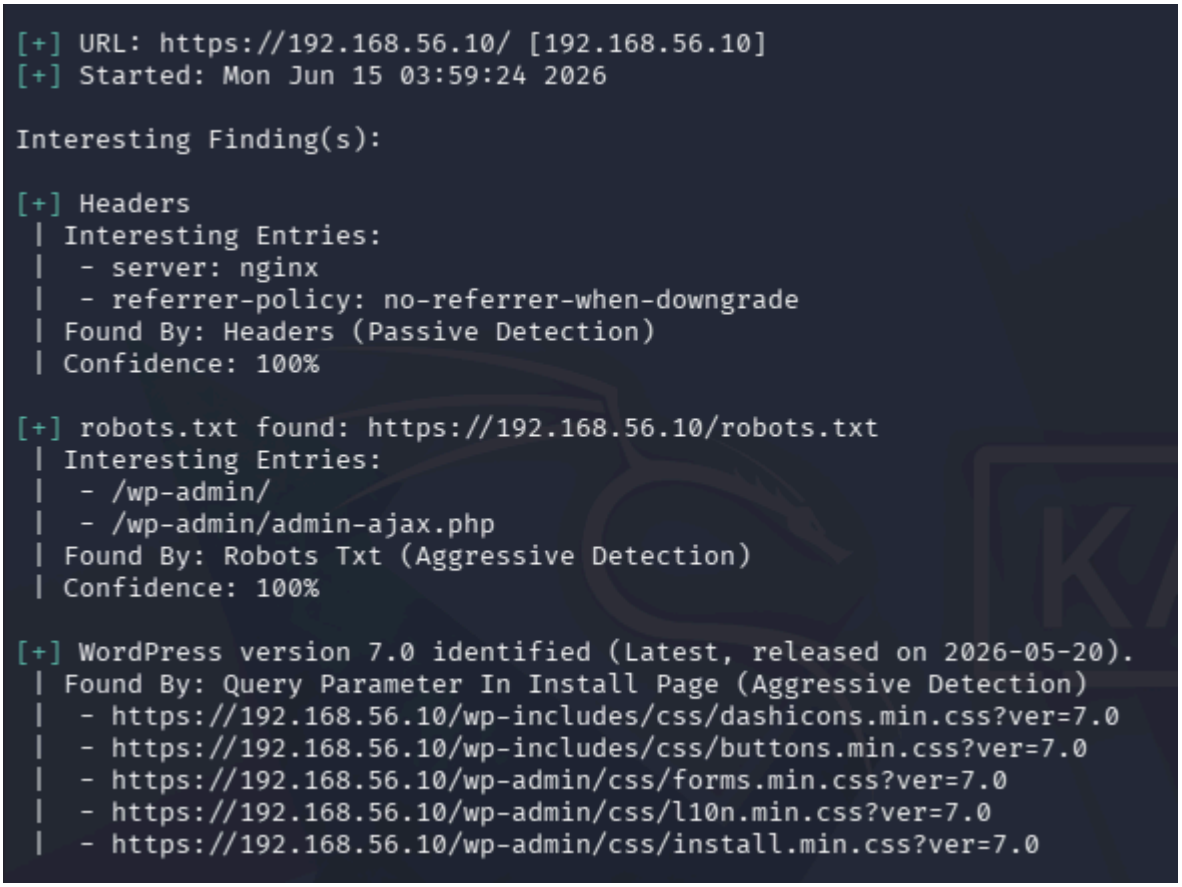


Figure 49: wpscan after hardening + correction 1

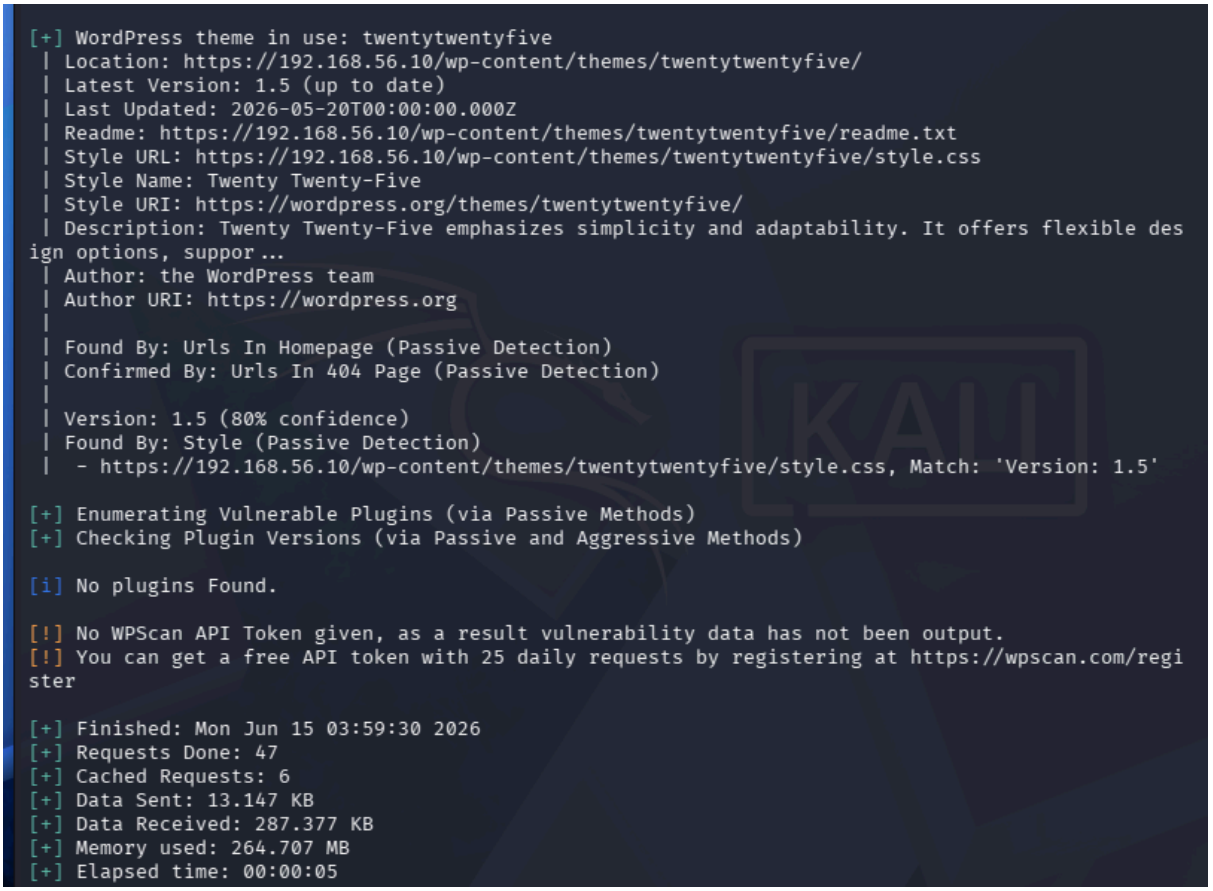


Figure 50: wpscan after hardening + correction 2

readme.html and wp-cron.php not visible any more. WordPress Version is still visible, but this time via CSS/JS query parameter and install page. This is harder to hide and could potentially risk a malfunction. Still the version is the latest one so there are no known vulnerabilities. This is why we decide to keep it this way.

Since we now have an active https, we also want to do a sslcan.



Figure 51: Sslscan via kali-user after hardening 1

only modern protocols are enabled, unsafe / old protocols disabled

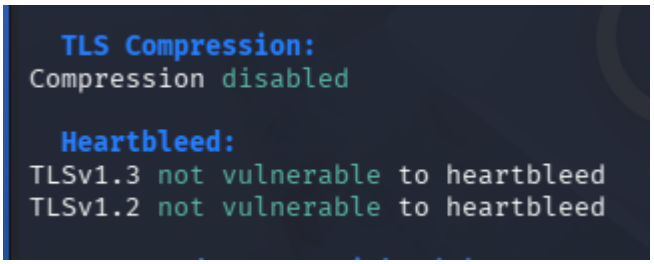


Figure 52: Sslscan via kali-user after hardening 2

tls compression disabled, no CRIME attacks
heatbleed not vulnerable, hearbleed is one of the best nown tls vulnerabilities, we are not affected

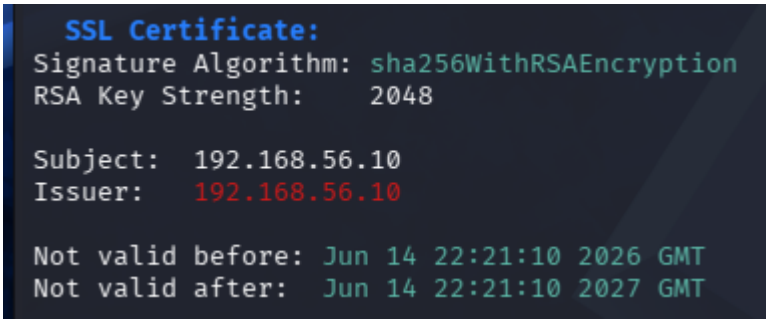


Figure 53: Sslscan via kali-user after hardening 3

rsa 2048 bit is used, that is long enough
issuer == subject, self-signed as discussed before
valid for a year

Let’s also do the lynis audit again. Fresh after Debian install it was 61, after installing all of the services it was 69.

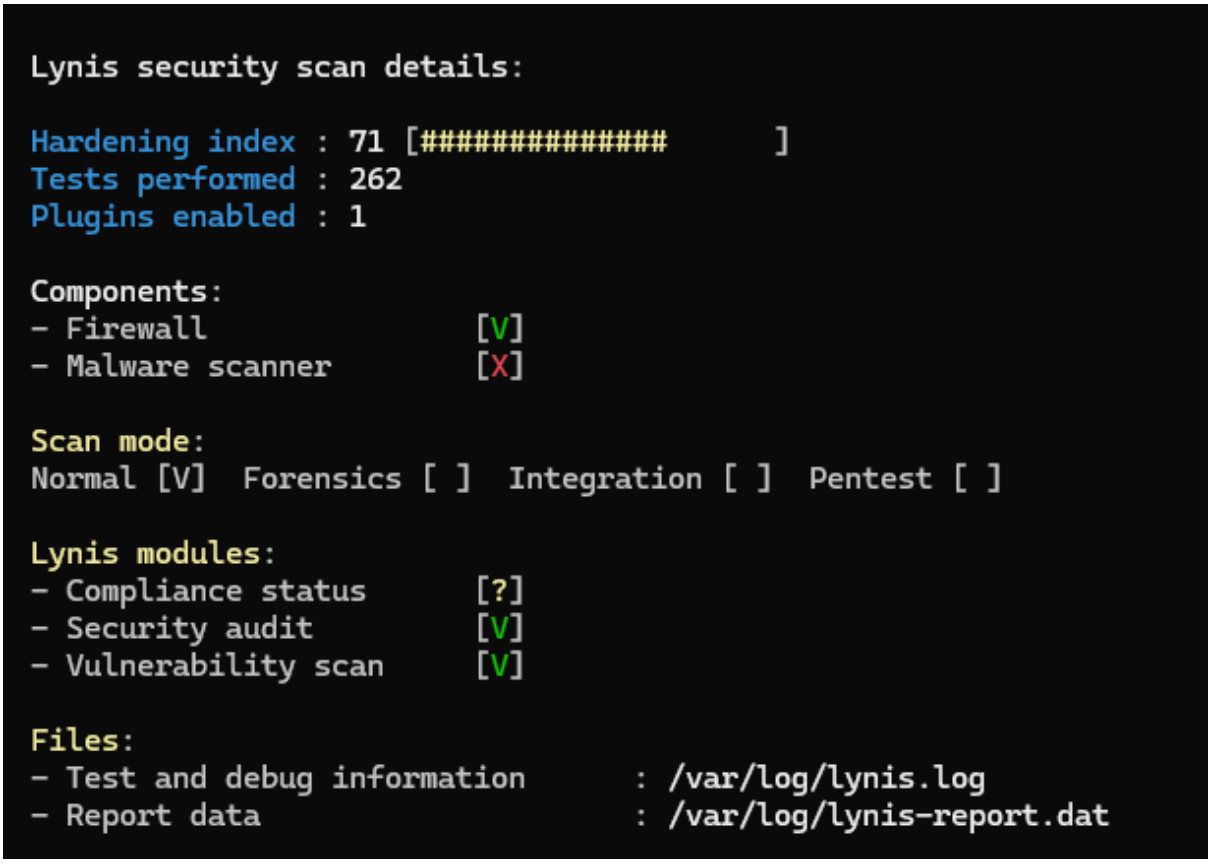


Figure 54: Lynis scan after hardening

Now we want to look at the suggestions:
sudo grep -A 2 'Suggestion' /var/log/lynis.log | head -60
The most important suggestions include:
In production we would also add a malware scanner which we don't use because it would need a lot of resources.
GRUB password is irrelevant for a vm, would be good to use in production.
While installing Debian, we decided to not separate partitions. Now it would be too hard to change it again. Also maybe we could use it in production.
We don't use pam because it is only relevant for users logging in with passwords. We use ssh. Same for how long a password is active before you have to change it.
We could deactivate core dumps and install needrestart.

echo '* hard core 0' | sudo tee -a /etc/security/limits.conf - no process is allowed to create core dumps
Needrestart sees which services have to be restarted after updates.

```
wp-viki@wp-server:~$ grep 'hard core' /etc/security/limits.conf
* hard core 0
wp-viki@wp-server:~$ needrestart -v
-bash: needrestart: command not found
wp-viki@wp-server:~$ sudo needrestart
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
```

Figure 55: Output needreset

Both work now too and we don't need to restart anything.

We also want to test our rate limits.

```
┌──(kali-user㉿kali)-[~]
└─$ ab -n 20 -c 5 -k https://192.168.56.10/wp-login.php
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

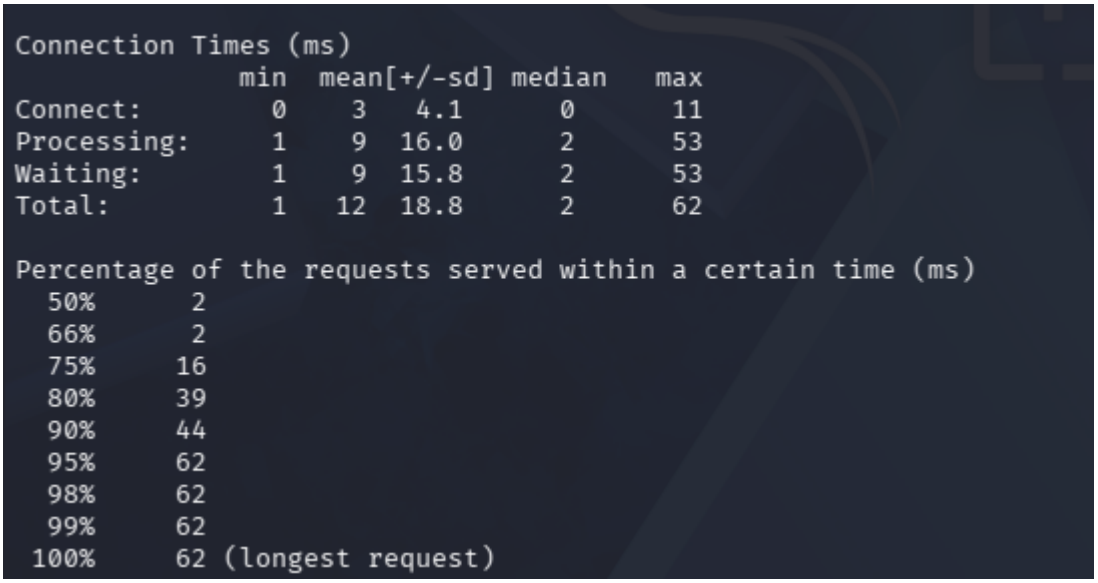
Benchmarking 192.168.56.10 (be patient).....done


Server Software:        nginx
Server Hostname:        192.168.56.10
Server Port:            443
SSL/TLS Protocol:       TLSv1.3,TLS_AES_256_GCM_SHA384,2048,256
Server Temp Key:        X25519 253 bits

Document Path:          /wp-login.php
Document Length:        4666 bytes

Concurrency Level:      5
Time taken for tests:   0.100 seconds
Complete requests:      20
Failed requests:        16
   (Connect: 0, Receive: 0, Length: 16, Exceptions: 0)
Non-2xx responses:      16
Keep-Alive requests:    16
Total transferred:      29272 bytes
HTML transferred:       21704 bytes
Requests per second:    199.54 [#/sec] (mean)
Time per request:       25.058 [ms] (mean)
Time per request:       5.012 [ms] (mean, across all concurrent requests)
Transfer rate:          285.20 [Kbytes/sec] received
```

Figure 56: Ab scan via kali-user 1



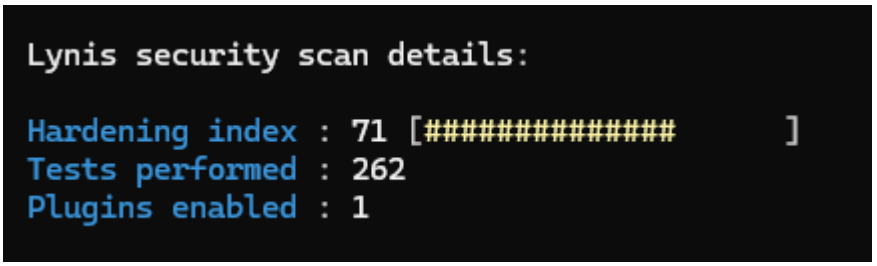
```
Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    3   4.1      0      11
Processing:     1    9  16.0      2      53
Waiting:        1    9  15.8      2      53
Total:          1   12  18.8      2      62

Percentage of the requests served within a certain time (ms)
  50%      2
  66%      2
  75%     16
  80%     39
  90%     44
  95%     62
  98%     62
  99%     62
 100%     62 (longest request)
```

Figure 57: Ab scan via kali-user 2

20 requests were sent, 16 failed and triggered 503, not 2xx responses. Only 4 requests went through (our burst 3 + 1)

After deactivating core dump and installing needrestart:



```
Lynis security scan details:

Hardening index : 71 [##############      ]
Tests performed : 262
Plugins enabled : 1
```

Figure 58: Final Lynis scan after hardening + correction

Still 71

To sum up what we did:

Area	Before	What we did	After
Open Ports	22, 80	UFW, no DB port exposed	22, 80, 443
HTTPS	not active	self-signed certificate	TLS 1.2/1.3 active
TLS Versions	not configured	ssl_protocols restricted	TLS 1.2/1.3 only

Nginx Version	visible	server_tokens off	not visible
Security Headers	none	add_header directives in nginx	HSTS, X-Frame options, X-content type options, Referrer policy
XML-RPC	accessible	nginx block	403 forbidden
wp-cron.php	accessible	nginx block	403 forbidden
readme.html	accessible	nginx block	403 forbidden
wp-config.php	accessible	nginx block	403 forbidden
PHP execution in uploads	executable	nginx block	403 forbidden
wp-admin access	unrestricted	IP restriction	only 192.168.56.0/24
WordPress version	visible via meta tag, RSS feed and JS/CSS quere parameters	remove meta tag via remove_action, removed RSS generator via add_filter	partially visible - meta tag and RSS feed removed, version still visible via JS/CSS query parameters, accepted risk as version is up to date
Login brute force	unlimited	rate limiting 5r/m, burst 3	16/20 requests blocked
Plugins	akismet, hello (inactive)	deleted via wp-call	none
Themes	3 installed, 2 inactive	2 deleted via wp-call	only active theme
File permissions	uncontrolled	755 dirs, 644 files, 640 wp-config.php	minimal necessary, no world-writable files or directories found
Audit logging	systemd only	auditd installed	wp-config.php, sshd_config, nginx config monitored
Lynis score	61 after debian install, 69 after os hardening	full hardening	71
Backup	none	mariadb-dump + tar	created and restore tested

3 Conclusions

We are a bit surprised that the final lynis score is almost not different to the second one. Still lynis tracks some things that we decided not to do on purpose or were not even able to do. No malware server, no external log server, no AppArmor profiles for Nginx and MariaDB (because they don't exist), no pam configuration for password complexity (because we use SSH), no IDS/IPS (because we used fail2ban as a lightweight version for that).

Additional to that, lynis is not the only metric that is important or can track our measurements. It doesn't track the Nginx security header, blocked php in uploads, wp-admin ip restrictions, auditd rules, ratelimits on wp-login.php or blocked xml-rpc. Lynis is better in tracking OS hardening, which we did while setting everything up mostly but it's not so goot for tracking WordPress specific measurements. We might even say that for this server the WordPress specific measurements are more relevant.

The Nginx hardening seems to have the biggest measurable effect. From no security headern, http/1.1 and visible versions and accessible sensitive data to a fully hardened web server.

Conflicts between security and functionality come up while trying to hide the WordPress version via JS/CSS query parameters. It would be more secure but could risk malfunctions. We accept that risk on purpose. Also DISALLOW_FILE_MODS is not active. It would block dashboard updates so we choose a compromise with the wp-cli update process.

Risks that are still there.

- no AppArmore profile for Nginx and MariaDB
- self-signed certificate instead of real Let's Encrypt
- WordPress version partially visible
- no malware scanner
- no remote logging, logs could be manipulated
- zero-days in WordPress core or theme

In production we would use Let's Encrypt with real domain, have a dedicated databank server and a log server for remote logging. We would also place a web application firewall and maybe a content delivery network in front of the server. With enough time and resources we would also write our own AppArmor profiles for Nginx and MariaDB. Malware scanner and IDS / IPS would be added too.

4 References

Course Materials

- Exercise 02 GNU Linux Server. FH Campus Wien, 2026.
- Exercise Hints and Scenarios. FH Campus Wien, 2026.
- Security Testing Tools. FH Campus Wien, 2026.
- Linux Server Hardening. FH Campus Wien, 2026.
- Linux Client Hardening. FH Campus Wien, 2026.

WordPress

- WordPress Hardening. https://developer.wordpress.org/advanced-administration/security/hardening/
- WordPress File Permissions. https://developer.wordpress.org/advanced-administration/server/file-permissions/
- WordPress Backups. https://developer.wordpress.org/advanced-administration/security/backup/

Nginx

- Nginx ngx_http_limit_req_module. https://nginx.org/en/docs/http/ngx_http_limit_req_module.html
- Nginx ngx_http_core_module. https://nginx.org/en/docs/http/ngx_http_core_module.html

MariaDB

- MariaDB Security Documentation. https://mariadb.com/docs/server/security/securing-mariadb

Debian

- unattended-upgrades manpage. https://manpages.debian.org/bookworm/unattended-upgrades/unattended-upgrade.8.en.html

Tools

- WPScan User Documentation. https://github.com/wpscanteam/wpscan/wiki/WPScan-User-Documentation
- Lynis Security Tool. https://cisofy.com/lynis/

5 Appendix

Complete Nginx config (/etc/nginx/sites-available/wordpress):

```
# Rate Limiting Zone
limit_req_zone $binary_remote_addr zone=wp_login:10m
rate=5r/m;

# HTTP -> HTTPS Redirect
server {
   listen 80;
   server_tokens off;
   server_name _;
   return 301 https://$host$request_uri;
}
```

```
# HTTPS Server
server {
   listen 443 ssl http2;
   server_name _;
   root /var/www/wordpress;
   index index.php;

   # TLS
   ssl_certificate /etc/ssl/certs/wordpress-selfsigned.crt;
   ssl_certificate_key
/etc/ssl/private/wordpress-selfsigned.key;
   ssl_protocols TLSv1.2 TLSv1.3;
   ssl_ciphers HIGH:!aNULL:!MD5;

   # Nginx Version verstecken
   server_tokens off;

   # Security Header
   add_header Strict-Transport-Security "max-age=31536000"
always;
   add_header X-Content-Type-Options "nosniff" always;
   add_header X-Frame-Options "SAMEORIGIN" always;
   add_header Referrer-Policy "no-referrer-when-downgrade"
always;

   # WordPress
   location / {
       try_files $uri $uri/ /index.php$args;
   }

   # PHP
   location ~ \.php$ {
      # PHP in Uploads blockieren
      if ($request_uri ~* "/wp-content/uploads/.*\.php$") {
          return 403;
      }
      include snippets/fastcgi-php.conf;
      fastcgi_pass unix:/run/php/php8.2-fpm.sock;
   }

   # XML-RPC blockieren
   location = /xmlrpc.php {
       deny all;
   }
```

```
    # wp-login Rate Limiting
    location = /wp-login.php {
        limit_req zone=wp_login burst=3 nodelay;
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.2-fpm.sock;
    }

    # /wp-admin/
    location /wp-admin/ {
       allow 192.168.56.0/24;
       deny all;
    }

    # Sensible Pfade blockieren
    location ~* \.(env|git|sql|bak|log)$ {
        deny all;
    }
    location = /wp-config.php {
        deny all;
    }

    # PHP in Uploads blockieren
    location ~* /wp-content/uploads/.*\.php$ {
        deny all;
    }

    # .htaccess blockieren
    location ~ /\.ht {
        deny all;
    }

    # readme not visible
    location = /readme.html {
       deny all;
    }

    # disable wp-cron access
    location = /wp-cron.php {
       deny all;
    }
}
```

auditd rules (/etc/audit/rules.d/wordpress.rules):

```
wp-viki@wp-server:~$ sudo cat /etc/audit/rules.d/wordpress.rules
-w /var/www/wordpress/wp-config.php -p wa -k wp_config_change
-w /etc/ssh/sshd_config -p wa -k sshd_config_change
-w /etc/nginx/sites-available/wordpress -p wa -k nginx_config_change
```

Figure 59: Final auditd rules

php-fpm config (changed parts):

```
listen = /run/php/php8.2-fpm.sock
;   static  - a fixed number (pm.max_children) of child processes;
;             pm.max_children      - the maximum number of children that can
;             pm.start_servers     - the number of children created on startup.
;             pm.min_spare_servers - the minimum number of children in 'idle'
;             pm.max_spare_servers - the maximum number of children in 'idle'
;             pm.max_spawn_rate    - the maximum number of rate to spawn child
;             pm.max_children           - the maximum number of children that
;             pm.process_idle_timeout   - The number of seconds after which
; forget to tweak pm.* to fit your needs.
pm.max_children = 5
pm.start_servers = 2
pm.min_spare_servers = 1
pm.max_spare_servers = 3
;pm.max_spawn_rate = 32
;pm.process_idle_timeout = 10s;
pm.max_requests = 500
;pm.status_path = /status
;pm.status_listen = 127.0.0.1:9001
request_terminate_timeout = 30s
; The timeout set by 'request_terminate_timeout' ini option is not engaged after
;request_terminate_timeout_track_finished = no
```

Figure 60: Changes parts in php-fpm config

MariaDB Grants (SHOW GRANTS FOR 'wp-db-user'@'localhost'):

```
+-----------------------------------------------------------------------------------------------------------------------+
| Grants for wp-db-user@localhost                                                                                       |
+-----------------------------------------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `wp-db-user`@`localhost` IDENTIFIED BY PASSWORD [redacted]                                      |
| GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, INDEX, ALTER ON `wordpress`.* TO `wp-db-user`@`localhost`         |
+-----------------------------------------------------------------------------------------------------------------------+
```

Figure 61: MariaDB Grants